18/1/19

Date :
Page No :

DBMS

Database is a collection of logically related data and their description.

Features of data in a database:

(i) Shared - shared between users and applications
(ii) Persistence - data should not get deleted in anyway
(iii) Validity - data should be correct
(iv) Consistency - data should remain consistent
(v) Non-redundant - data should not get replicated anywhere
(vi) Independent -

Ex. College database

DBMS → Data + Software

DBMS is a collection of inter-related data and a set of programs to access the data. The collection is called database.
The primary use is to store and retrieve data from database that is easy and efficient.

Applications:

- (i) Enterprise - sales of products, purchase information
  (ii) Accounting - payment receipts, balance
  (iii) Human Resources - information of the employees
  (iv) Manufacturing - units manufactured, dispatched
  (v) Online Retailers - eg. flipkart, amazon

- (i) Banking and Finance - accounts
  (ii) Credit card transactions - payments, billings
  (iii) Finance - holidays, sales, policies

Scanned by CamScanner

Date :
Page No :

- (i) Universities - student, staff, peons, clerks
- (ii) Airlines - flight reservations, ticketing
  They were the first to use DBMS.
- (i) Telecompanies - balance in account, bills, recharge plans

Purpose of Database System:

(i) Data redundancy and inconsistency
(ii) Difficulty in accessing data
(iii) Data isolation
(iv) Integrity problems
(v) Atomicity problems - failure of system midway during transactions
(vi) Concurrent access anomalies - multiple users can't access data simultaneously
(vii) Security problems - providing access as per requirement

File management	DBMS
• It is used in small systems. (C++ is used)	It is used in vast systems. (Oracle is used)
• It is cheap.	It is expensive.
• It has simple structure.	It has complex structure.
• Low level design	High level design
• Security is missing.	Very secure
• Single user	Multiple users
• Isolated data app1 — data1 app2 — data2	Shared data
• Simple backup mechanism	Complex backup mechanism

Scanned by CamScanner

Date :
Page No :

View of Data:

DBMS provides an abstract view of data i.e. hides unnecessary information.

There are 3 views:

(i) Physical level - how has the data been stored

(ii) Logical level - what all data has been stored

(iii) View / External level - providing only required portion of view

Instances and Schemas:

Schema is overall design of database. It is constant.

Instances are no. of records in a database. There can be many instances in a database.

Subschema - Part of whole schema

Datamodels:

1. Relational model - Data is maintained using tables (called relations).

Eg.

Sid	Sname	Sclass
1	A	I
2	B	III
3	C	IV

Relation

Columns - attribute

Row - tuple

attributes

tuples

Above is an example of a record based model. It contains 3 records.

2. Entity relationship model - ER model

ER diagrams are formed and then reduced to tables.

3. Object Based model - It is object oriented.

4. Object Relational Data model - Relational + Object Based

Scanned by CamScanner

Date :
Page No :

5. Semi-structured data model - used in XML

21/1/19

Database languages

Data definition language (DDL)
- Create
- Alter
- Drop (deleting the table)

Data manipulation language (DML)
- Insert
- Update
- Delete (deleting the contents of table)

DML

Procedural DML - (telling the software how to retrieve data by writing code)

Non-Procedural DML (only data matters; backend calculations don't matter)

Non-Procedural DML

Formal Query language
eg. Relational algebra, Tuple calculus

Commercial Query language
eg. SQL

DCL (Data Control language): Grant and revoke data

DQL (Data Query language): Select data

Transaction languages: eg. commit, savepoint, rollback

Data independence: Based upon the levels

(i) Physical independence (ii) Logical independence

Scanned by CamScanner

Date :
Page No :

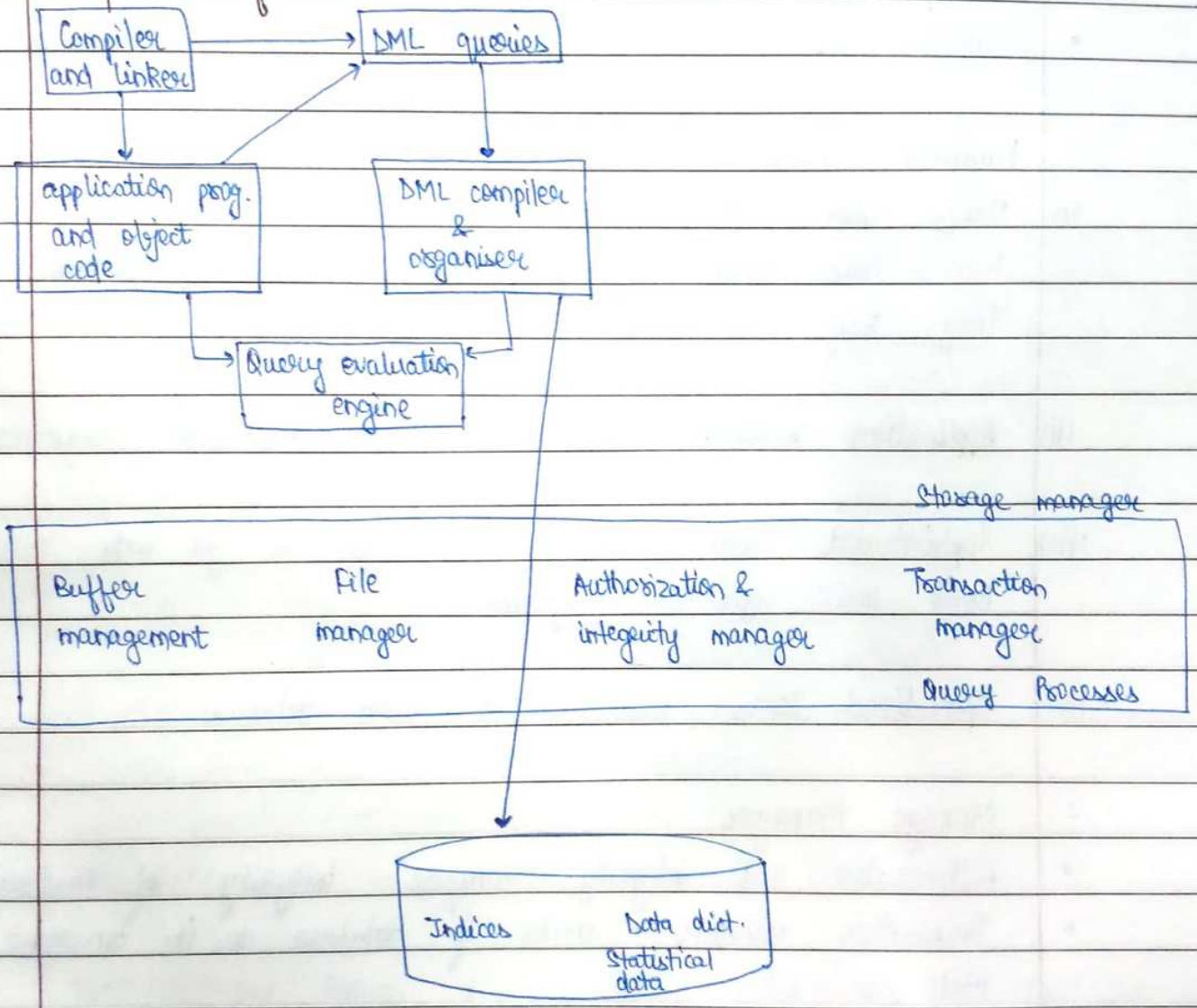
Physical independence

- Physical storage structure and devices can be changed without affecting external schema
- Done to improve performance
- Provides immunity to logical or external level

Logical independence

Conceptual schema can be changed without affecting external schema
Structure of database is altered.
Provides immunity to external level

Components of Database :



Scanned by CamScanner

÷ DBA (Database Administrator):
Person in charge of database

Functions:
- Defining conceptual database - schema definition
- Storage structure and access method definition
- Security and integrity checks - authorization of data
- Backup and recovery strategy
- Granting user access

÷ Database Users:

(i) Naive user - Unsophisticated users
Don't have much information about backend of database
They only use it

(ii) Application Programmers - one who writes application programs

(iii) Sophisticated Users - write the queries to get data
Don't write application programs

(iv) Specialized Users - well versed with database

÷ Storage Manager:
- Authorization and integrity manager - integrity of constraints
- Transaction manager - checks if database is in consistent state or not
- File manager - determines space allocation, format of files
- Buffer manager - how much data will be stored at various locations

Scanned by CamScanner

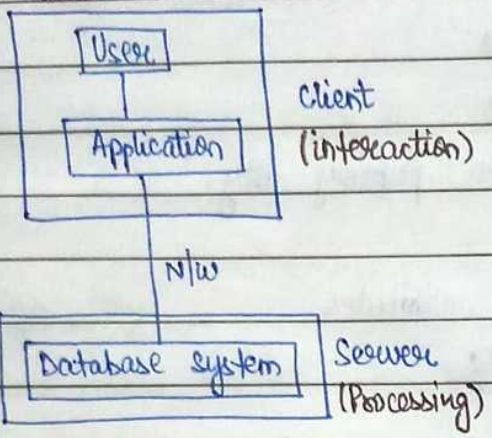
Date :
Page No :

÷ Query Processor:

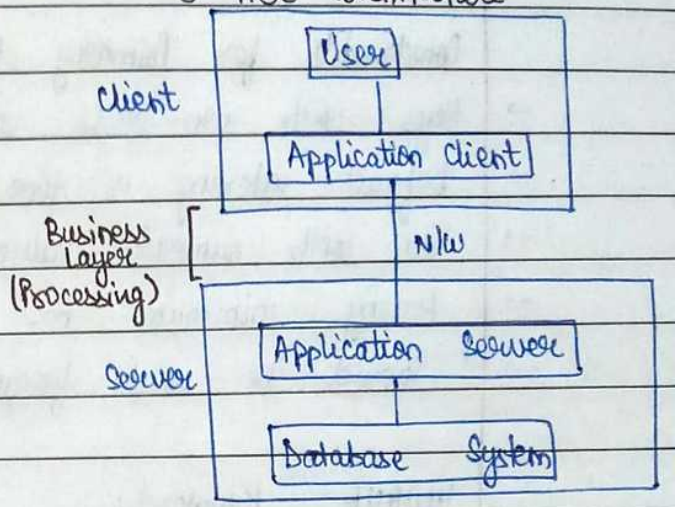
- DDL compiler - compiles data definition files
- DML interpreter - manipulating data ; query optimization
- Query evaluation engine - executes the instructions / queries
-

÷ DBMS architecture:

2 - tier architecture



3 - tier architecture



- No stability as system fails if no. of users increases
- More stable

25/1/19

Sid	Sname	Branch
S1	A	CSE
S2	B	IT
S3	C	IT

→ Relational instance / Tuple

→ attributes

Cardinality is no. of rows in a table.

Key: A key allows us to identify a set of attributes and differentiate tuples of relation.

1. Primary Key: any one of the candidate keys
2. Candidate Key: which uniquely identifies a relation
3. Alternate Key: candidate keys which are not primary
4. Secondary Key: key from candidate keys other than primary

Scanned by CamScanner

Date :
Page No :

5. Super Key: superset of all candidate keys
6. Foreign Key: borrowed key from another set of attributes
7. Composite Key: combined candidate keys

Prime attributes: Attributes belonging to candidate keys
Non-Prime attributes: not belonging to candidate keys

Constraints for Primary Keys:
→ Keys with non-NULL value
→ Default indexing is done on primary key.
→ Go with numeric value.
→ Having minimum no. of attributes
→ Should be used frequently

UNIQUE Keyword:

```
CREATE TABLE Student (Sid varchar(10), Sname varchar(20)
NOT NULL, PPNo varchar(20) UNIQUE, License No varchar(10)
UNIQUE NOT NULL, DOB date NOT NULL, Fname varchar(10)
NOT NULL, Unique (DOB, Fname));
```

1. The max. no. of superkeys for the relation schema ~~are~~ R(E,F,G,H) with E as the key is $2^{n-1} = 2^3 = 8$
(candidate)

2^{n-k} → n → total no. of attributes
k → no. of key attributes

2. For $R(A_1, A_2, \dots, A_n)$, total no. of superkeys is 2^{n-1}
$CK = A_1$

Super keys = A_1, A_1A_2, $A_1A_2A_3$, ... (n-1) times

3. For $R(A_1, A_2, \dots, A_n)$, total no. of superkeys is 2^{n-2}
$CK = A_1A_2$

Scanned by CamScanner

Date :
Page No :

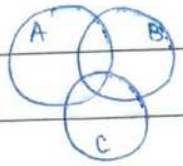
4. If $CK = (A_1, A_2 A_3)$, no. of superkeys will be $2^{n-1} + 2^{n-2} - 2^{n-3}$

$SK(A_1) = 2^{n-1}$ $SK(A_2 A_3) = 2^{n-2}$ $SK(A_1 A_2 A_3) = 2^{n-3}$

5. Given the $CK = A_1 A_2, A_3 A_4$, no. of superkeys will be $2^{n-2} + 2^{n-2} - 2^{n-4}$

6. For $R(A, B, C, D, E)$, no. of superkeys will be $2^{n-1} + 2^{n-1} + 2^{n-1} - 2^{n-3}$

$CK = (A, B, C)$



Foreign Key:

Sid1	Name	Login
S_1		
S_2		
S_3		
S_4		

Referenced relation

Sid2	Cid	Fee
S_1		
S_1		
S_2		
S_3		

Referencing relation

Sid1 should uniquely identify the relations.
Sid1 is foreign key for the second table.

```
CREATE TABLE Enrolled (Sid varchar(10), cid varchar(10), fee
number, PRIMARY KEY (Sid, Cid), Foreign Key (Sid) references
student (Sid) ON DELETE NO ACTION);
```

Foreign key is also called the referential key.

Self referential Key: referencing is within the table

Scanned by CamScanner

Date :
Page No :

Q. A foreign table has 2 attributes A and C, where A → PK and C → FK referencing A with ON DELETE cascade.
The set of all tuples that must be additionally deleted to preserve referential integrity when the tuple (2,4) is ~~defined~~ deleted is 4

Referential integrity constraint:

ON DELETE Cascade → deleted from both tables

ON DELETE No action

" " Set NULL → value in parent table would be set to NULL

A	C	
2	4	×
3	4	
4	3	
5	2	×
7	2	×
9	5	×
6	4	

28/1/19

Entity Relationship model (E-R model)

In high level database design, E-R diagrams are used.
Entity is an object which can be differentiated from other objects in the real world.

Entity set is set of similar entities.

Multi - valued attributes :

Sid	Name	Course (multi - valued)
1	A	C/C++
2	B	C++ / Java

Scanned by CamScanner

Date:
Page No.:

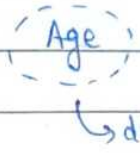
Compound attributes: name



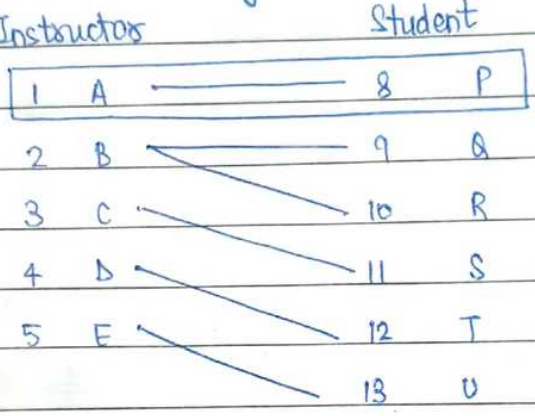


Derived attributes:





Relationship: association among entities



The function played by an entity in a relationship is called its role.

Recursive relationship set: when an entity participates more than once in a relationship
eg. B is instructor to both Q and R.

Binary relationship: involves 2 entity sets

Degree of relationship set: No. of entity sets participating in a relationship set
A relationship set is represented by ◇.

Relationship set constraints: (i) Total (ii) Partial

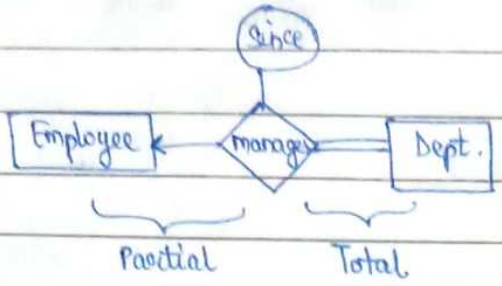
Student	Subject
A	C
B	C++

Every student has to pick 1 subject.
This is total relationship set participation.

Scanned by CamScanner

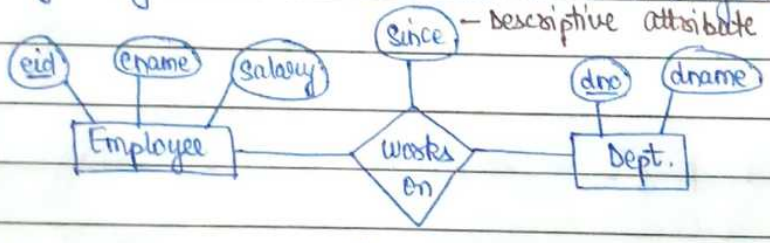
Date :
Page No :

In case of choosing class representative, it would be partial participation.



E_1 —— 10/10 —— d_1
E_2 —— 11/10 —— d_2
E_3 —— 12/10 —— d_3
E_4

Primary Key is underlined in an E-R diagram.

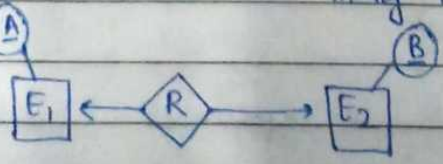


Create table workson (eid varchar(10), dno varchar(5), since date, FOREIGN KEY (eid) reference employee ON DELETE CASCADE);

Mapping cardinalities or cardinality ratio:
Cardinality means how an entity set is related to another entity set.

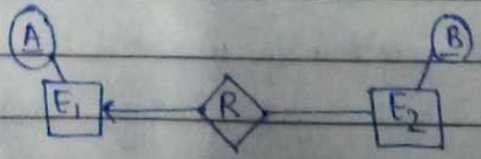
1. One to one
2. one to many
3. many to one
4. many to many

1. One to one



Candidate = (AB)
Key

2. One to many

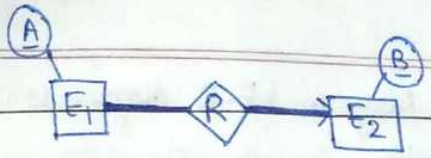


CK = B

Scanned by CamScanner

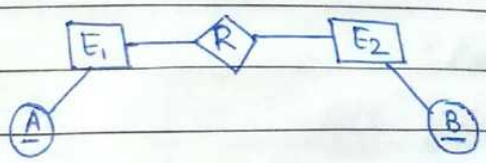
Date :
Page No :

3. Many to one



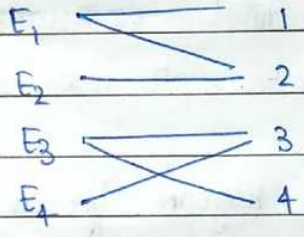
CK = A

4. Many to many

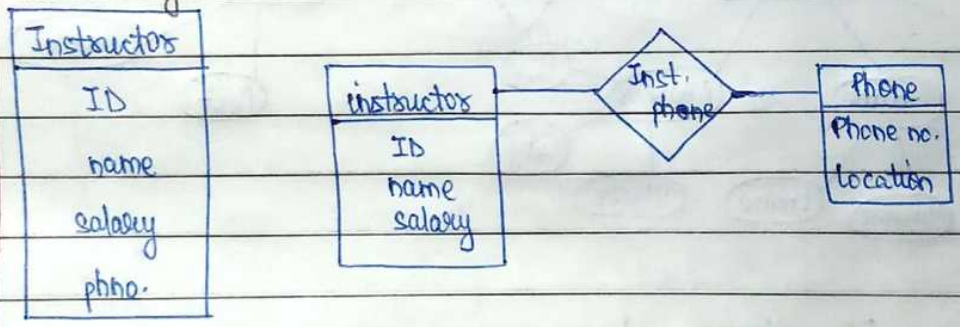


CK = (AB)

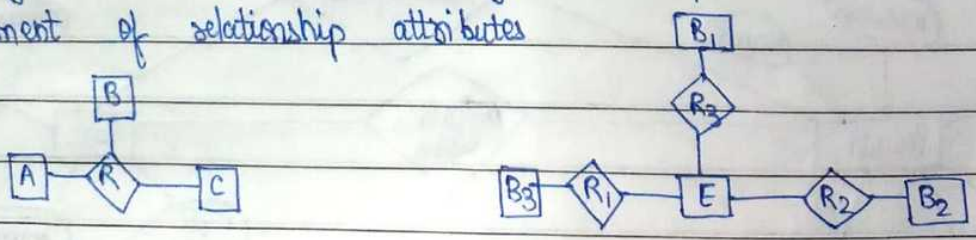
One to one : eg. HOD and department
One to many : eg. HOD and faculty
Many to one : eg. Students and college
Many to many : eg. Emp Project



ER Design Issues:



1. Use of entity set v/s attributes
2. Use of entity set v/s relationship set
3. Binary relationship v/s n-ary relationship
4. Placement of relationship attributes



Ternary Relationship

n-ary Relationship

Scanned by CamScanner

Date :
Page No :

1/2/19

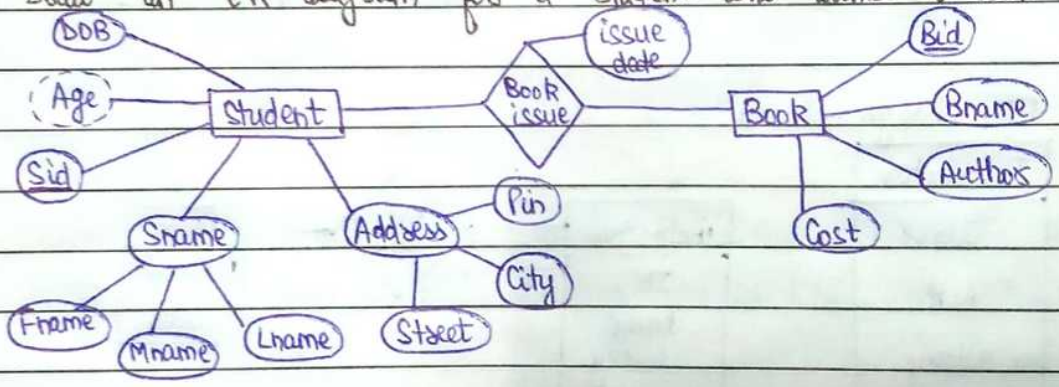
Entity Relationship Diagram (E-R diagram):
Used to express the overall structure of the database graphically

Major exponents:

1. Rectangle - entity sets
2. Ellipse - attributes
3. Diamonds - relationships
4. Lines - links
5. Double ellipse - multi-valued attributes
6. Dashed ellipse - derived attributes
7. Double lines - total participation
8. Double rectangle - weak entity sets (which don't have PRIMARY KEY)

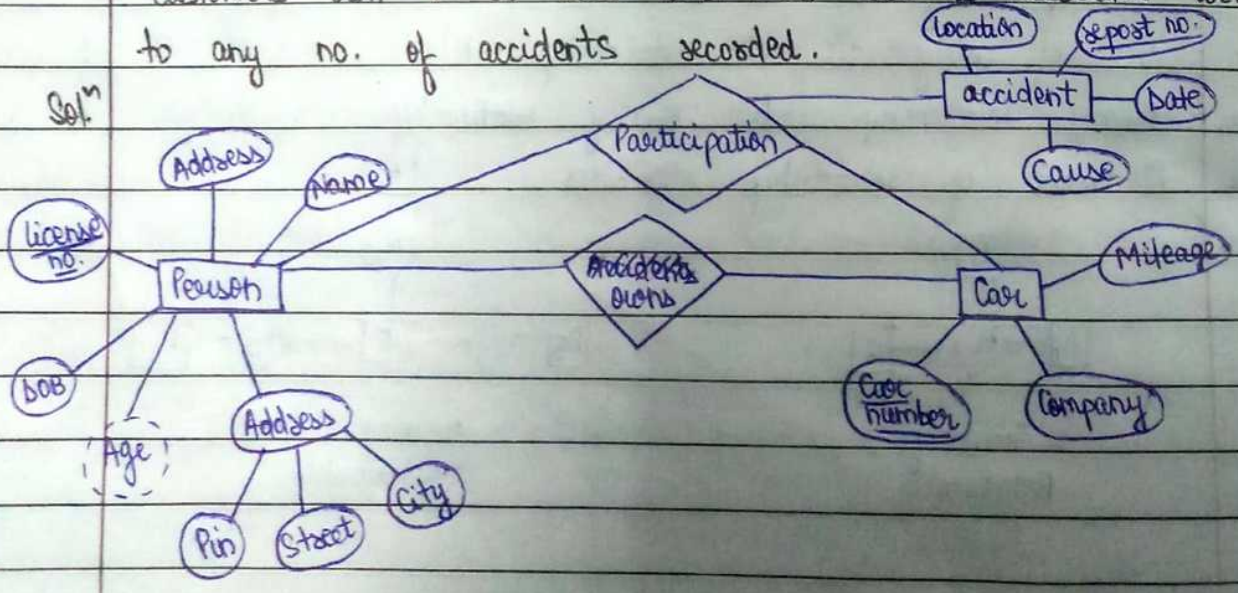
Q. Draw an ER diagram for a student who wants to issue a book.

Sol^n



Q. Draw an ER diagram for a car insurance company whose customers own 1 or more car. Each car is associated with a 0 to any no. of accidents recorded.

Sol^n



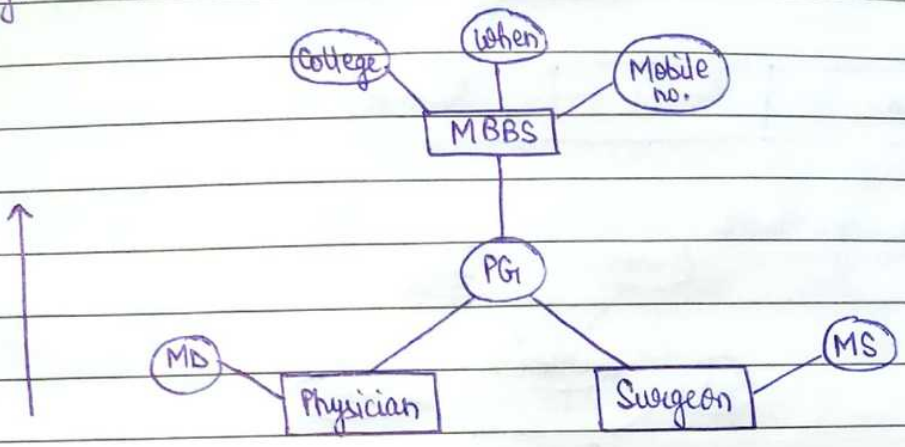
Scanned by CamScanner

Date :
Page No :

Extended ER features:

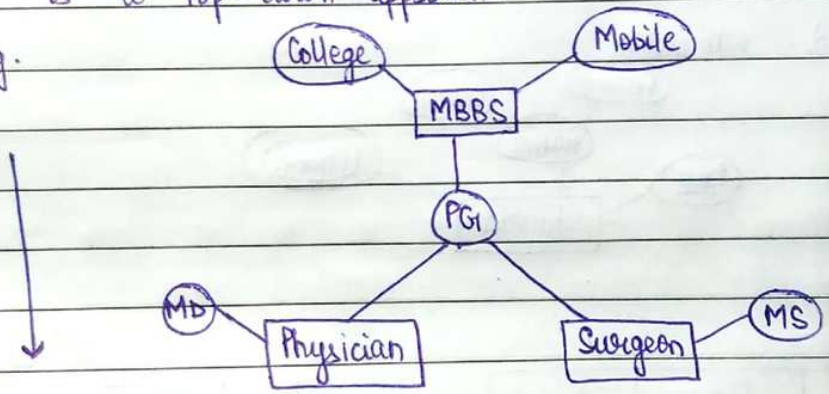
1. Generalization: Abstracting view of object as a single general class
It is a bottom-up approach.

Eg.



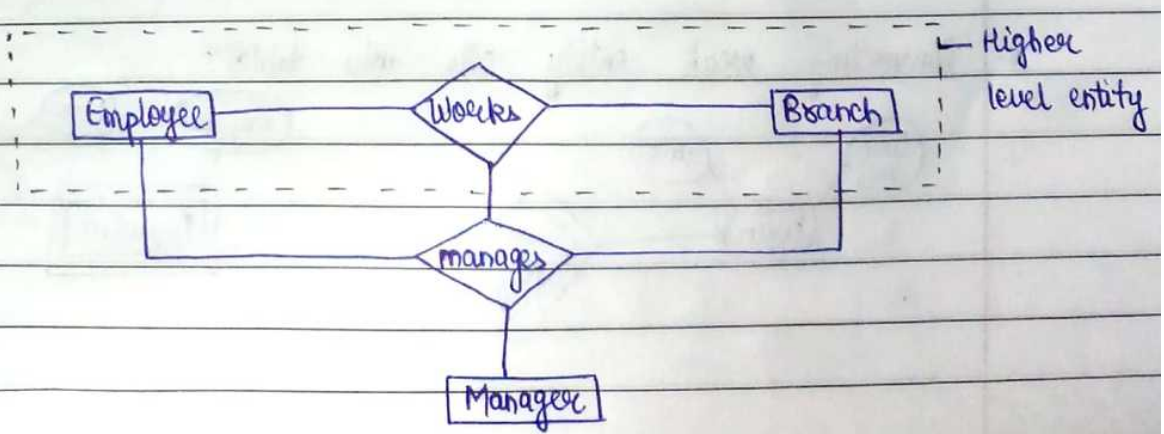
2. Specialization: Introducing new characteristics to an existing class of objects to create new classes
It is a top-down approach.

Eg.



3. Aggregation: Compiling information on an object thereby abstracting higher level of object.

Eg.

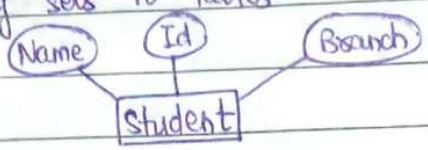


Scanned by CamScanner

Date :
Page No :

1. Converting ER diagrams into tables:

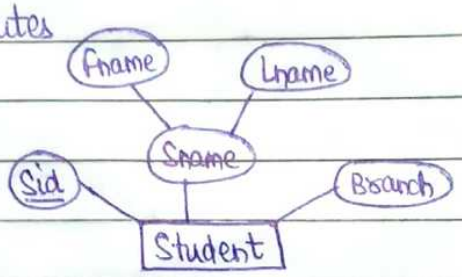
1. Converting entity sets to tables



Student

Name	Id	Branch

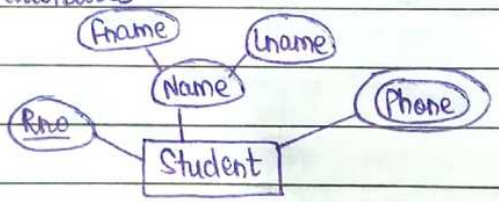
2. Composite attributes



Student

Sid	Fname	Lname	Branch

3. Multi-valued attributes



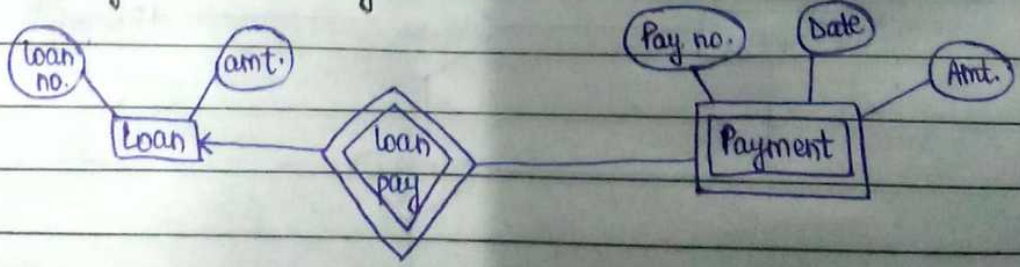
Student

Rno	Fname	Lname

Rno	Phone

- A weak entity set does not have a primary key. It has a partial key.

2. Converting weak entity sets into tables:



Scanned by CamScanner

Payment

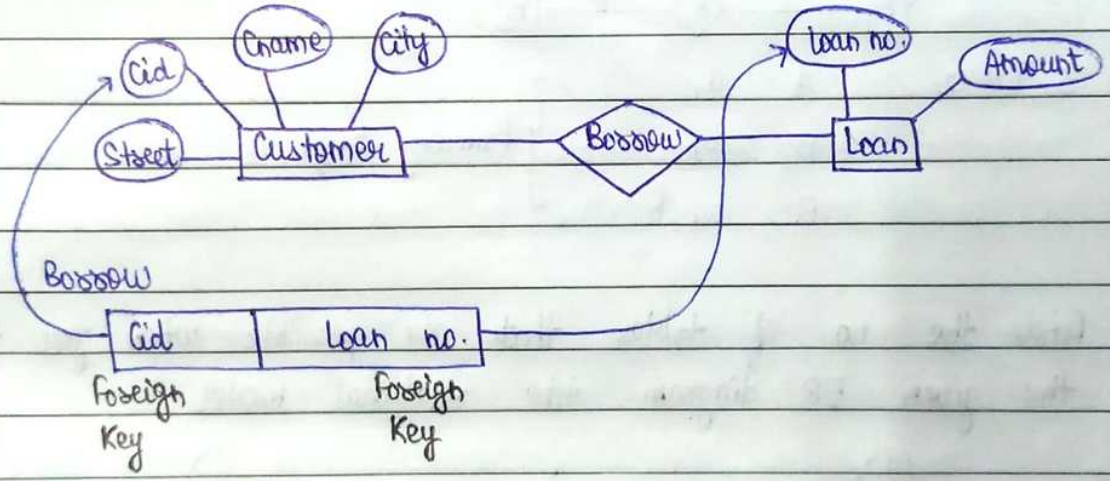
Loan no.	Payment no.	Date	Amount

Their combination acts as primary key.

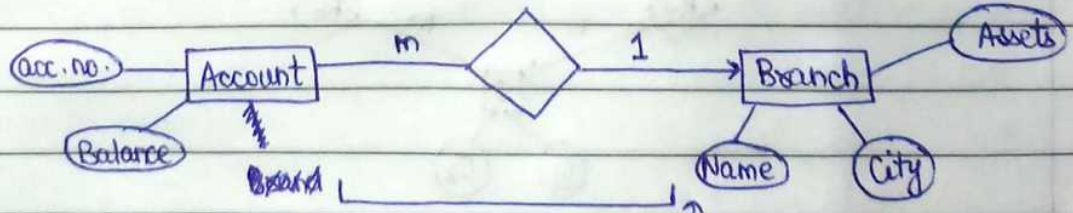
- There is an owner entity set corresponding to a weak entity set.
- Cardinality is either one-to-many or many-to-many.
- D/B table should always be combined with weak entity set.
- We can't make a relationship b/w 2 weak entity sets.

3. Converting relationship sets to tables:

(i) many-to-many:



(ii) one-to-many / many-to-one:



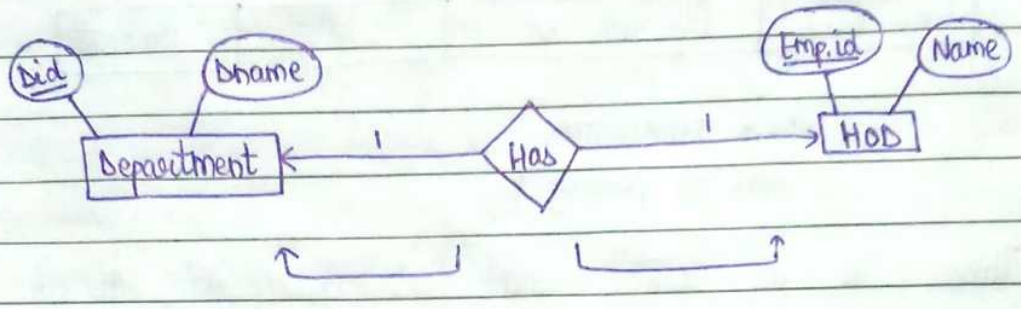
Account

Acc. no.	Balance	Name (FK)

Scanned by CamScanner

Date :
Page No :

(iii) one-to-one:



Department

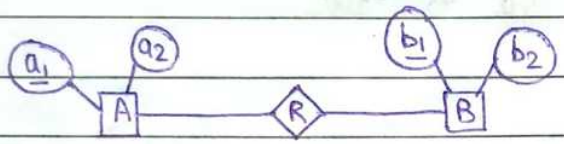
Did	Dname	Emp.id

either of the 2 should be mentioned

HOD

Emp.id	Name	Did

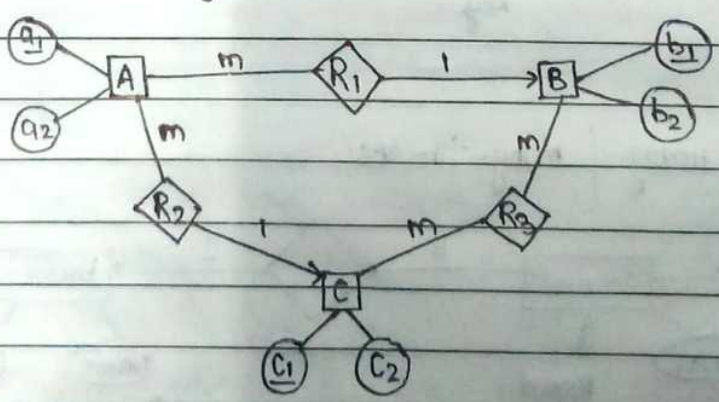
Eg.



3 Tables

A	a_1	} Primary Keys
B	b_1	
R	a_1, b_1	

Q. Find the no. of tables that are possible when you translate the given ER diagram into relational model.



Sol.n

C ($\underline{c_1}$, c_2)

B ($\underline{b_1}$, b_2)

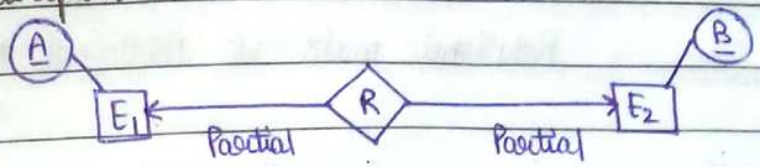
R_3 ($\underline{b_1}$, $\underline{c_1}$) FK FK

AR1R2 ($\underline{a_1}$, a_2, $\underline{b_1}$, $\underline{b_2}$)
PK

Scanned by CamScanner

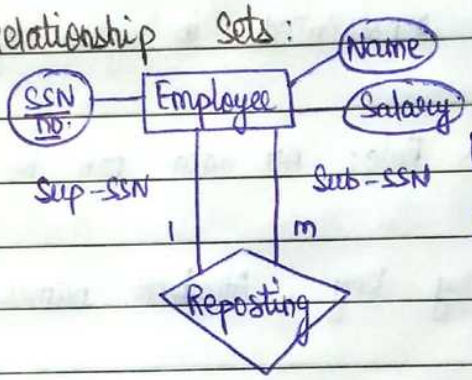
Date :
Page No :

Partial Participation:



- 3 tables will be created (E_1, R, E_2).
- If there is total participation, all 3 tables converge into 1 (E_1RE_2).
- If there is total participation on one side and partial on another side, either E_1R or E_2R is created.

Self - Referential Relationship Sets:



Employee has a relationship with himself.

Table created will be (Sub-SSN, Sup-SSN)
FK FK

Scanned by CamScanner

Date :
Page No :

2/2/19

Unit - 2

Relational model v/s Relational DBMS

→ Relational model is concerned with what is required which separates it from how the model is implemented.

→ represents data in form of table

Codd's Rule for DBMS:

1. Information Rule: All the data must be presented in form of tables i.e. all the data represented in form of rows and columns.

2. Guaranteed access Rule: All data can be accessed without any ambiguity.
Tablename, Primary Key, Attribute names

3. Systematic treatment of NULL values: A field should be allowed to remain empty or NULL is different from empty string.

4. Dynamic online catalog based on relational values:

5. The comprehensive data sublanguages: Relational data must support 1 language (DDL, DCL and DML). That language must support all this.
DML - Data Manipulation Language
DCL - Data Control Language
DDL - Data Definition Language
- Data authorization
- View definition
- Integrity constraints
- Transaction control

6. View Updating Rule: Changes must be updated at all levels.

Scanned by CamScanner

Date :
Page No. :

7. High level insert, update and delete: Selected data → operation performed eg. nested queries

8. Physical data independence: Changes at physical level w/o affecting the upper and higher levels (logical and external levels)

9. Logical data independence:
Changes at logical level w/o changing external level
eg. Bank, IRCTC

10. Integrity independence: Constraints that languages use must be supported.

11. Distribution independence: User does not know about the database present at a place.

12. Non - subversion Rule: Database security should not be subversed or weakened.

Difference b/w DBMS and RDBMS:

DBMS	RDBMS
• does not support client server architecture	supports client server architecture
• does not support distributive databases	mostly supports distributive databases
• No tight security of data	Multiple levels of security such as logging at operating system level, command level, object level.
• Supports less than 7 codd's rules	Supports more than 7 codd's rules
• Speed of operation is low.	High speed of operation
• less need for hardware and software	More need for hardware and software

Scanned by CamScanner

Date :
Page No :

DBMS	RDBMS
• Platform used is DOS	Platforms used are DOS, UNIX, VMS
• 3GL 3rd generation language	4GL 4th generation language
• Facilities and utilities are limited.	Facilities and utilities are high.

Constraints:

1. The Domain constraint: Every attribute has a specific domain means if char type data is there, then that attribute will only have characters.

2. Key constraint: Each table must have a key value of key ≠ NULL

3. Entity integrity: If we have entities then some relationship is established among them using keys (value of key ≠ NULL).

4. Referential integrity: In case of foreign keys, value of foreign key referencing other table should not be NULL.
No. of tuples in F.K. and referenced key should be same.

5. Informational constraint: Customer (Cust_name, Cust_ID)
Order (order ID, Cust_ID, order date)

6. Semantic Integrity Constraints: eg. Salary of employee does not exceed manager's salary.

7. Functional Dependency Constraints: Functional relationship b/w constraints or b/w 2 attributes

Scanned by CamScanner

Date :
Page No :

6/2/19

Relational Algebra :

1. Select operation: denoted by σ (equivalento to WHERE in SQL)

Syntax : $\sigma_{(cond.)}(R)$ → tablename

Operators used are $<, >, <=, >=, =, \neq$.

Connectors used are $\wedge, \vee, \neg$.

eg. $\sigma_{B_1}(\sigma_{B_2}(R))$, $\sigma_{B_2}(\sigma_{B_1}(R))$

eg. Instructor:
ID name dept-name salary building

To select dept-name = Physics , $\sigma_{dept_name = 'Physics'}(instructor)$

$\sigma_{salary > 9000}(instructor)$

To find dept. whose name is same as building name ,

$\sigma_{dept_name = building}(instructor)$

2. Project operation : denoted by π

Used to retrieve data from specific attributes

Syntax : $\pi_{A1, A2, A3, ---}(R)$

eg. To list all instructor IDs, name, salary but do not care about dept-name , $\pi_{ID, name, salary}(instructor)$

To find the name of all instructors in the Physics dept.,

$\pi_{name}(\sigma_{dept_name = 'Physics'}(instructor))$

3. Set operations :

(i) UNION - The 2 relations should be union compatible i.e. no. of attributes and domain must be same in both relations. The domain of ith attribute of R_1 and i^{th} attribute for R_2

Scanned by CamScanner

must be same for all i.

R

id	name	age
1	A	12
2	B	13

S

id	name	age
3	C	14
4	D	15

R ∪ S

id	name	age
1	A	12
2	B	13
3	C	14
4	D	15

To show the names of people in R or S,

$\pi_{name}(R) \cup \pi_{name}(S)$

(ii) INTERSECTION:

R

id	name	age
1	A	12
2	B	13
3	C	14

S.

id	name	age
3	C	14
4	D	15

$\pi_{name}(R) \cap \pi_{name}(S)$,

R ∩ S

id	name	age
3	C	14

(iii) SET DIFFERENCE:

Student

id	name
1	A
2	B
3	C

Fail

id	name
1	A
2	B

Scanned by CamScanner

Date :
Page No. :

To show students who have failed, $\pi_{name}(student) - \pi_{name}(Fail)$

To find courses taught in fall 2009 not in spring 2009,

Section:

Course_id	Semester	Year
–	–	–
–	–	–

$\pi_{course_id}(\sigma_{semester='fall' \wedge year='2009'}(section)) -$

$\pi_{course_id}(\sigma_{semester='spring' \wedge year='2009'}(section))$

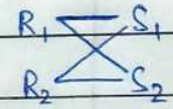
4. Cartesian Product: denoted by X

It is not used much. It is used for joining of 2 tables.

$R \times S$

R	S
R_1	S_1
R_1	S_2
R_2	S_1
R_2	S_2

R S



R_1	R_2
!	!
x	y

$R_1 \times R_2 \rightarrow (x+y)$ attributes

(xy) tuples

2 different relations

To find names of students having grade 'A',

Student

name	sno
A	1
B	2
C	3

Grade

sno	grade
1	A
2	C
3	B

$\pi_{name}(\sigma_{grade='A'}(Student \times grade))$

Scanned by CamScanner

Date :
Page No :

5. Rename operation: To rename attributes, relations

To find names of students with grade 'A',

$\rho_{Student\,grade}(\sigma_{grade='A'}(student \times grade))$

↳ renaming

ρ_{a_1/a_2}

a_1 — new attribute name

a_2 — old attribute name

6. Join operation: denoted by $\bowtie$

Join

Inner Join

1) Theta Join ($\bowtie_\theta$)

<, >, <=, >=

2) Equi Join ($\bowtie_=$)

=

3) Natural Join ($\bowtie$)

based on common attributes among tables

Outer Join

1) Left Join ($\bowtie$)

tuples from left table take part

2) Right Join ($\bowtie$)

tuples from right table take part

3) Full Join ($\bowtie$)

tuples from both tables take part

Inner Join:

Mobile		Laptop	
Model	Price	Model	Price
Nokia	10k	Dell	30k
Samsung	20k	Acer	20k
iphone	50k	Lenovo	10k

Theta Join: To purchase both mobile and laptop but mobile price should be less than laptop price,

mobile $\bowtie_{(mobile.price < laptop.price)}$ laptop

Scanned by CamScanner

Date :
Page No :

Equi Join: To purchase s.t. prices of both are same,

mobile $\bowtie_{(mobile.price = laptop.price)}$ laptop

Natural joins form a cartesian product of 2 arguments perform selection enforcing equality on those attributes that appear in both the relational schema and replace duplicate attributes.

Emp.

Name	ID	D-name
A	1	Finance
B	2	Sales
C	3	Sales
D	4	CSE

Dept.

D-name	Manager
Finance	M_1
Sales	M_2
Marketing	M_3

Natural Join: Emp $\bowtie$ Dept

Name	ID	D-name	Manager
A	1	Finance	M_1
B	2	Sales	M_2
C	3	Sales	M_2

Outer Join:

Left outer Join: Emp $\bowtie$ Dept

Name	ID	D-name	Manager
A	1	Finance	M_1
B	2	Sales	M_2
C	3	Sales	M_3
D	4	CSE	NULL

Right outer Join: Emp $\bowtie$ Dept

Name	ID	D-name	Manager	
A	1	Finance	M_1	
B	2	Sales	M_2	
C	3	Sales	M_3	
Null	Null	Mkt.	M_4	(suppose)

Scanned by CamScanner

Date :
Page No :

Full outer Join: No null values

11/2/19

7) Division operation: $R_1(x) = R_1(z) \div R_2(y)$
Result contains all tuples appearing $R_1(z)$ in combination with $R_2(z)$

R_1

Course #	Name
10	B.Tech
12	MBA
14	MCA

R_2

Course #
10

$R_1 \div R_2 \longrightarrow$

Name
B.Tech

To find names of students who failed in all the subjects,
Student:

Name	Sub-failed	Subject
A	DBMS	DBMS
A	JAVA	JAVA
B	DBMS	
C	JAVA	

Student ÷ Subject ⟶

Name
A

Query will be π_{name} (student ÷ subject)

8) Generalised Projection: denoted by π
It is applied on a particular list.
eg. credit info (cust-name, limit, credit)
To find the difference of limit and credit for each customer,

$\pi_{cust-name, limit-credit}$ (credit info)

Scanned by CamScanner

9.) Aggregate functions: SUM, MIN, MAX, AVERAGE, COUNT, COUNT DISTINCT

To find the sum of salaries of all instructors,

$\mathcal{G}_{sum(salary)}$ (instructors)

It is denoted by $\mathcal{G}$.

To find the total no. of instructors who teach a course in the spring 2010 semester,

$\mathcal{G}_{count\ DISTINCT\ (ID)}$ ($\sigma_{semester = 'spring' \wedge year = 2010}$ (instructors))

10.) Assignment operator:

To find the names of all customers who have their accounts in all the branches in Delhi,

$R_1 \leftarrow \pi_{branchname}$ ($\sigma_{branchcity = 'Delhi'}$ (Branch))

$R_2 \leftarrow \pi_{customername,\ branchname}$ (Depositor $\times$ Account))

$R_2 \div R_1$

To find the average salary in each department,

$\pi_{dept_name}\ \mathcal{G}_{average\ (salary)}$ (instructors)

To find the average salary of all instructors,

$\mathcal{G}_{average\ (salary)}$ (instructors)

Multiset Relational Algebra:

R_1		R_2
A	B	2
1	a	3
2	a	3

Scanned by CamScanner

Date :
Page No :

$\Pi_B(R_1) = a, a$

$\Pi_B R_1 \times R_2$ a 2
a 3
a 3
a 2
a 3
a 3

SQL

- High-level language
- Free format syntax

Number (4, 2) eg. 24.02
↓ ↳ digits after decimal
total digits

varchar2 (size)

Commands:

DDL	DML	DCL
CREATE	SELECT	GRANT
ALTER	UPDATE	REVOKE
DROP	INSERT	
	DELETE	

CREATE TABLE Dept (deptname varchar (20) NOT NULL, building varchar (15) UNIQUE, budget numeric (12, 2), primary key (deptname));

CREATE TABLE instructor (ID varchar (5), name varchar (20) NOT NULL, dept_name varchar (20), salary numeric (8, 2), primary key (ID), foreign key (dept_name) references (dept));

Scanned by CamScanner

Date :
Page No :

CHECK (fieldname) in ('M', 'F')

DESC <table name> ;

ALTER constraint:

→ To add new columns
→ To add new integrity constraints
→ To modify existing columns (↑se or ↓se length)
→ To change the datatype
→ To drop integrity constraints

Q. Create a table supplier with the given attributes:
id number 3
name varchar 20
s-code number 4

1. Add new columns pincode and city to the table.
2. Change the width of pincode to 5.
3. Add a primary key after creating the table.
4. Remove the primary key
5. Add not NULL
6. Drop not NULL
7. Add constraint CHECK to deposit column
8. Drop CHECK constraint
9. Add a primary key constraint
10. Drop a primary key constraint

Sol.ⁿ
```
CREATE TABLE Supplier
(id number(3), name varchar(20), s-code number(4));
ALTER TABLE Supplier
ADD ( pincode varchar(3), CITY varchar(5));
ALTER TABLE Supplier
MODIFY ( pincode varchar(5)), ADD PRIMARY KEY (ID));
ALTER TABLE Supplier DROP Primary Key;
```

Scanned by CamScanner

Date :
Page No :

```
ALTER table supplier MODIFY Name NOT NULL;
ALTER table supplier MODIFY Name NULL;
ALTER table supplier ADD CONSTRAINT CKDEP
CHECK ( deposit between 4000 and 5000);
ALTER table supplier DROP CKDEP;
ALTER table supplier ADD constraint PID Primary Key (ID);
ALTER table supplier DROP constraint PID;
```

Entering values interactively from keyboard:

```
INSERT INTO CUSTOMER VALUES
(&Cust-no, '&Lname', '&fname');
```

Concatenation operator:

```
Select fname || ' ' || lname from customer;
```

15/2/19

Sorting in SQL:

```
Select Lname, fname, Pin, State from Cust order by State Desc;
```

```
Select upper (state) from Cust;
```

To retrieve all rows where the first name contains the word 'Raj',

```
Select from Emp where fname like '%Raj%';
                                 'Raj_';
                                 'Raj%';
```

```
Update Cust Set Pin = "10001" where ID='2';
```

Views

↳ view_name

```
Create view EView as
Select empno, ename, dname from emp, dept where
emp.deptno = dept.deptno;
```

Scanned by CamScanner

Date :
Page No :

- If primary key is not included in a view, we can't insert values in view.

- ORDER BY clause can't be used with CREATE view and the view can't be updated. It can't be used with JOIN, DISTINCT

Number functions: ABS, COS, COSH, SIN etc.
Select Abs (200) from Dual;

Aggregate functions: MIN, Max etc.

Count functions: Count (*), Count DISTINCT, Count
↓ counts NULL as well
↓ removes duplicates
↳ does not count NULL

Character functions: INITCAP, LENGTH etc.

Conversion functions: TO_CHAR (__) → converts into character
NVL → used to insert a value in place of NULL but only for display purpose; does not make changes to table

Decode functions: Select City Decode ('UP' Uttar Pradesh, 'BNG' Bangalore) from City;

Date functions: Adding dates, months b/w 2 dates
To_date (__) → converts to date-type
Savepoint S1;
Delete from customer where state = 'DEL';
Rollback; // undo
Savepoint S2;
Rollback to S1;

Scanned by CamScanner

Date :
Page No :

- Select Emp-No, Sum (Basic) from Sal group by Emp-no order by Sum(Basic);

ID	Emp-No	Basic	Total Deduction
	1001	2000	
	1002	1000	
	1003	3000	
	1001	2000	

Output:

Emp-No	Sum
1002	1000
1003	3000
1001	4000

- Select S.Emp-No, Emp-Name, Avg (Basic) from Salary S, Emp
  ↳ alias name
  where S.Emp = Emp.Emp-No
  group by S.Emp-No, Emp-Name Having Avg (Basic) >= 5500
  order by Avg (Basic);

Sub-Queries (Nested Queries)

It is a query within a query.
It must return only 1 column.
The table used at different levels should be same.
'Between' operator can't be used.

To display the employees who earn less than the average salary in the organisation,
Select Avg (Basic) from Salary ; → for avg. salary

Select * from Salary where Basic < (Select Avg (Basic) from Salary);

Scanned by CamScanner

Date :
Page No :

To display names of employees not working in mkt. dept.,
Select E-Name from Salary where Dept not in (Select emp-no, from salary where Dept = 'MKT');

To select employee names where deduction < 500,
Select * from employee where (Select emp-name from employee where deduction < 500);

To display names of employees from CSE dept.,
Select ename from employee where exists (Select * from salary where emp.emp-no = salary.emp-no and Dept. = 'CSE');

- 'Exists' is used when we use joins in nested queries.

Reports

Generating a salary report in SQL Plus,

```
SET PAGESIZE 80
SET LINESIZE 75
TTITLE CENTER 'SALARY REPORT'          (By default - left align)
BTITLE 'END OF SALARY REPORT'          ( " - center " )
Column Emp-Name Format A10 TRUNC
Column 'Net Sal' Format 9,999.00
Select S.Emp-No, Emp-Name, Basic, Commission, Deduct, Basic +
Commission - Deduction 'Net Sal' from Salary S, Emp where
S.Emp-No = Emp.Emp-No;
TTITLE OFF
BTITLE OFF
CLEAR COLUMN
SET FEEDBACK ON
```

Scanned by CamScanner

18/2/19

PL/SQL - Superset of SQL

It is a block structured language.
There are mainly 2 blocks

Anonymous Block
- does not have any name
- executed only once

Name Block
has a name
executed as many times the code runs
Sub programs
/
Trigger Procedures / function
cursor

DECLARE - declaration block
variables, cursors, user defined exceptions

BEGIN
SQL and PL/SQL statements

EXCEPTION
Trap the error

END;
BEGIN and END are compulsory blocks.

eg. A program to add 2 no.'s

```
Set serveroutput on;        // to display the o/p
DECLARE
i integer := 10;
j integer := 20;
c integer;
BEGIN
```

Scanned by CamScanner

Date :
Page No :

```
c := i+j;
DBMS_OUTPUT.PUT_LINE(c);
END;
/
```

DBMS_OUTPUT → Package name; PUT_LINE → Procedure name

// to run the code

Package contains procedures:

Q.1. Write a PL/SQL code to retrieve the employee name, join basic and designation of an employee whose no. is i/p by the user.

Q.2. Write a PL/SQL to run a for loop in reverse.

Sol^n 2.

```
Set serveroutput on;
BEGIN
DBMS_OUTPUT.PUT_LINE('for loop ...');
FOR I IN REVERSE 1...10 LOOP
DBMS_OUTPUT.PUT_LINE('LOOPINDEX' || I);
END LOOP;
END;
/
Set serveroutput off;
```

|| → concatenation operator

```
IF condition:
IF (condition)
THEN ——
ELSE IF ——
END IF;
```

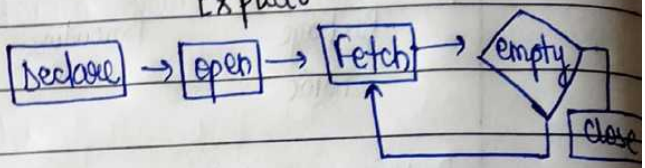
CASE syntax:

```
CASE expression WHEN exp1 THEN ——
                WHEN exp2 THEN ——
                |
                ELSE DEFAULT —
                END;
```

Scanned by CamScanner

Date :
Page No :

Cursor:
It is a pointer to the context area.
There are 2 types of cursors:
Implicit
Explicit



%. Found - returns True if row is found
%. Not found
%. IS OPEN - to check if cursor is open or not
%. ROW COUNT - It is 0 when ~~row count~~ cursor is open

```
DECLARE
CURSOR < Cursor Name>
IS < Select Statement >
```

To open:
```
OPEN < Cursor Name>
FETCH < Cursor Name > INTO Var 1, Var 2
        or
FETCH < Cursor Name> INTO < Record Name>
CLOSE < Cursor Name>
```

Q. Write a PL/SQL code to calculate the total and % of student marks in 4 subjects with the schema given below.
STUDENT (RNO, S_1, S_2, S_3, S_4, Total, Percentage)

Sol:
```
DECLARE
T NUMBER;
PER NUMBER;
CURSOR C1 IS
```

Scanned by CamScanner

Date :
Page No. :

```
SELECT * FROM STUDENT ;
REC C1 %.ROWTYPE;
  ↳ record variable
BEGIN
OPEN C1;
LOOP
  FETCH C1 INTO REC;
  EXIT WHEN C1 %.NOT FOUND;
  T:= REC.S1 + REC.S2 + REC.S3 + REC.S4 ;
  PER := T/4;
  UPDATE STUDENT SET TOTAL=T , PERCENTAGE = PER WHERE
  RNO = REC.RNO ;
  END LOOP ;
  CLOSE C1 ;
END;
```

```
CURSOR C1 is Select empno , ename from Emp;
V-empno EMP . eno %. Type;
V-ename EMP. ename %. Type ;
BEGIN
   OPEN C1;
   LOOP
      FETCH C1 INTO V.eno , V.ename;
      EXIT WHEN C1 %. NOT FOUND;
      DBMS_OUTPUT. PUT_LINE (V.eno || ' ' || V.ename)
      END LOOP;
      CLOSE C1;
END;
```

Scanned by CamScanner

Triggers:

- Triggers are required to enforce rules that can't be coded through referential integrity.
- To maintain an audit of changed values in data
- To derive column values
- To enforce certain business rules

CREATE [OR REPLACE] Trigger (Trigger name) [Before | After]
[DELETE | insert | update] [of column] on [User].table
[For each row]
[when condition]
[PL/SQL statements]

- Triggers are associated with a table and are automatically when the user performs insert/ update/ delete on the table.
- There can be many triggers on a table.
- There can be only 1 trigger of a particular type.
- Only 1 table can be specified in the triggering statements
- Trigger code can be typed in SQL+, Notepad.
- Inside the trigger, the correlation names the new/old to refer to data on cmd line.

Q. Write a code to create a table so that no operation can be performed on the employee table on Tuesday.

Sol^n:

```
CREATE TRIGGER EM_TUE
Before insert OR update OR delete on EMP;
Begin
    If RTRIM (UPPER (TO_CHAR (SYSDATE, 'DAY')) = 'TUESDAY'
    Then RAISE_APPLICATION_ERROR (-20101, 'Can't use table
     on Tuesday');
    ENDIF;
END;
```

(-20101 ↓ error no.)

Scanned by CamScanner

Date :
Page No :

To start trigger, START EM_TUE END;
To drop trigger, DROP trigger < >;
To disable, ALTER TABLE EMP Disable ALL TRIGGER;
To enable, After Table EMP Enable All Trigger;

PL/SQL Procedure:
Procedure / function is a named block.

IN OUT INOUT
Pass and return values

Q. Write an SQL procedure to update salary of the emp. no. passed as parameter.

Sol^n
```
Create [or Replace] Procedure
raisesal (E.no emp.empno %Type)
IS  ↳ Procedure name
BEGIN
    UPDATE EMP
    Set Sal = Sal+1000 where empno = eno;
    END raisesal;
// Calling a procedure
raisesal (10);
        ↳ argument
DROP raisesal;          // dropping procedure
```

Q. Write a PL/SQL function to return the square of a given no.

Sol^n
```
Create [or replace] Function SQR (N NUMBER)
RETURN NUMBER;
          ↳ return datatype
IS
BEGIN
    Return N*N;
    END;
/
```

Scanned by CamScanner

```
Set serveroutput on;
BEGIN
DBMS_OUTPUT.PUT_LINE("Square is" || SQR(5));
END;
/
Set serveroutput off;
```

PLSQL code to implement a function having table columns as parameter

```
CREATE [or replace] Function findsal (eno emp.empno %.Type)
Return NUMBER
IS
V.Sal emp.sal %.Type
Begin
Select sal into V.Sal from emp
where empno = eno;
Return V.Sal;
END;
```

8/3/19

Functional dependency

It is an association b/w 2 attributes of the same table.

A	B
1	4
1	5
2	8
3	9

not unique
∴ no functional dependency exists

Q. Consider a relation

A	B	C
1	2	3
4	2	3
5	3	3

Which of the following F.D. is incorrect?

(a) $A \rightarrow B$ correct

(b) $B \rightarrow C$ correct

Scanned by CamScanner

(c) $BC \rightarrow A$ incorrect

$2,3 \rightarrow 1$

$2,3 \rightarrow 4$

$3,3 \rightarrow 5$

(d) $AC \rightarrow B$ correct

Classification of F.D.:

1. Trivial functional dependency: F.D. $X \rightarrow Y$ is trivial iff Y is a subset of X. ($Y \subseteq X$)

2. Non Trivial functional dependency: It exists if there is atleast 1 attribute in RHS that is not a part of LHS.

$AB \rightarrow BC$

3. Full functional dependency: An attribute Y is fully functional dependent if it is functionally dependent on X but not on proper subset of X.

eg. $AB \rightarrow C$ ✓

$A \rightarrow C$ ✗

$B \rightarrow C$ ✗

C depends on combination of A and B. It does not depend on just A or just B.

4. Partial functional dependency: Given a relation R with functional dependency F defined on attributes of R and K is a candidate key and X is a proper subset of K and if $F: X \rightarrow Y$ then A is said to be partially dependent on K.

5. Transitive functional dependency: If you have $X \rightarrow A$ and $A \rightarrow Y$, then $X \rightarrow Y$. Y is transitively dependent upon X.

(Non key attributes) $\rightarrow$ (Non key attributes)

6. Candidate functional dependency: It is an F.D. which includes

Scanned by CamScanner

all the attributes of the table.
Dependency diagram consists all F.D.s of our database.

7. Primary functional dependency: in case of primary key
(Key attributes) → (Non key attributes)

8. Boyce Codd F.D. : (Non key attributes) → (Key attributes)

9. Multi valued F.D. : When 2 or more multi-valued facts about the same attribute occur in the same table.

B, C A
A →→ B
A →→ C

10. Joint F.D. : It exists if a relation R is the projection of X,Y,Z
X, Y and Z are projections (small tables) of R.

Closure Properties / Inference rules / Armstrong axioms:

1. The Reflexivity rule: If $Y \subseteq X$, then $X \to Y$
eg. $AB \to A \Rightarrow A \subseteq AB$

2. Augmentation: If $X \to Y$, then $XZ \to YZ$
augmentation

3. Transitivity: If $X \to Y$ and $Y \to Z$, then $X \to Z$.

4. Union: If $X \to Y$ and $X \to Z$, then $X \to YZ$.

5. Decomposition: If $X \to YZ$, then $X \to Y$ and $X \to Z$.

6. Pseudo transivity: If $A \to B$, $BC \to D$, then $AC \to D$

Scanned by CamScanner

Date :
Page No. :

Closure of a set of F.D. : (F^+)

F^+ is a set of all F.D. that can be determined using the given set of F.

F^+ is a set of all F.D. logically implied by F.

Total $(F.D.)^+ = 2^{2n}$ $\quad n \rightarrow$ no. of attributes

eg. F^+ for $R(AB) = 2^{2(2)} = 16$

Q. You are given a relation R and the attributes are R(ABCDE). F.D. are {$AB \rightarrow C$, $CD \rightarrow E$, $DE \rightarrow B$}. Find closure. Is AB a candidate key? If not, is ABD ? Explain.

Sol^n $AB^+ = ABC \rightarrow$ not CK as all attributes are not covered.

$ABD^+ = ABDCE \rightarrow$ CK as all attributes are covered

Closure of attributes (X^+)

Set of all attributes that can be determined using the given attributes X.

eg. $A \rightarrow B$

$B \rightarrow C$

$A^+ = ABC$ $\qquad$ $B^+ = BC$

Applications of attribute closure :

1. Finding keys of a relation :

If X^+ contains all attributes of a relation, then it is called superkey of R.

$X \rightarrow$ superkey of R

Q. You are given a relation R with attributes ABCDE and the set of F.D. as $A \rightarrow BC$, $CD \rightarrow E$, $B \rightarrow D$, $E \rightarrow A$. Find the candidate keys of R.

Scanned by CamScanner

Date :
Page No :

Sol^n $A^+ = ABCDE \rightarrow CK$

$B^+ = BD$

$C^+ = C$

$D^+ = D$

$E^+ = EABCD \rightarrow CK$

A and E are candidate keys. No need to check for AE^+.

$BC^+ = BCDEA \rightarrow CK$

$CD^+ = CDEAB \rightarrow CK$

$BD^+ = BD$

∴ There are 4 candidate keys in total.

2. Find additional functional dependency:

Q. You are given a relation R and F.D. are $A \rightarrow BC$, $B \rightarrow CD$, $D \rightarrow AB$. Find if $AD \rightarrow C$ is possible or not. (R(ABCD))

Sol^n $AD^+ = ADBC$

$AD \rightarrow \boxed{C}$ is getting covered in AD^+.

So, $AD \rightarrow C$ is possible.

Q. You are given a relation R and ABCDE are attributes.

$A \rightarrow B$, $CD \rightarrow E$

$A \rightarrow C$, $B \rightarrow D$ and $E \rightarrow A$

Which of the following F.D. is not implied by below set?

(i) $CD \rightarrow AC$

(ii) $BD \rightarrow CD$

(iii) $BC \rightarrow CD$

(iv) $AC \rightarrow BC$

Sol^n $CD^+ = CDEAB$ It includes AC so it is correct.

$BD^+ = BD$ Not implied

$BC^+ = BCDEA$ It is correct.

$AC^+ = ACBDE$ It is correct.

Scanned by CamScanner

9/3/19

Date :
Page No :

÷ How to check ~~independent~~ equivalent F.D.'s :

Q. $F = \{A \to C,\ AC \to D,\ E \to AD,\ E \to H\}$ Check if F and G are equivalent.
$G = \{A \to DC,\ E \to A\}$

Sol.ⁿ

F: $A \to C$	G: $A \to DC$
$AC \to D$	$E \to A$
$E \to AD$	
$E \to H$	
$A^+ = DC$	$A^+ = ACD$
$E^+ = EADC$ X	$E^+ = EADHC$
$AC^+ = ACD$	

∴ These 2 FDs are not equivalent.

Q. $F = \{A \to C,\ AC \to D,\ E \to AD,\ E \to H\}$
$G = \{A \to DC,\ E \to AH\}$

Sol.ⁿ

F: $A \to C$	G: $A \to DC$
$AC \to D$	$E \to AH$
$E \to AD$	
$E \to H$	
$A^+ = ADC$	$A^+ = ACD$
$AC^+ = ACD$	$E^+ = EADHC$
$E^+ = EAHDC$	

$F \subseteq G$ and $G \subseteq F$

∴ F and G are equivalent.

Q. $F = \{AB \to CD,\ B \to C,\ C \to D\}$
$G = \{AB \to C,\ AB \to D,\ C \to D\}$

Sol.ⁿ

F: $AB^+ = ABCD$	G: $AB^+ = ABCD$
$B^+ = BCD$ X	$C^+ = CD$
$C^+ = CD$	

Scanned by CamScanner

Date :
Page No :

$G \subseteq F$ but $F \not\subseteq G$ so they are not equivalent.
$F > G$ as $F \not\subseteq G$.

Q. Consider 2 sets F and G with FDs as below:
$F = \{ A \to B,\ AB \to C,\ D \to ACE \}$
$G = \{ A \to BC,\ D \to AE \}$

Sol.ⁿ
F: $A^+ = ABC$
$AB^+ = ABC$
$D^+ = DAEBC$

G: $A^+ = ABC$
$D^+ = DACEB$

∴ F and G are equivalent.

Q.
F: $A \to B$
$B \to C$
$AB \to D$

G: $A \to B$
$B \to C$
$A \to C$
$A \to D$

Sol.ⁿ
F: $A^+ = ABCD$
$B^+ = BC$
$AB^+ = ABCD$

G: $A^+ = ABCD$
$B^+ = BC$
They are equivalent.

Standard Set / Minimal Cover / Canonical cover

A set of FDs is minimal if it satisfies given points:

1. Every dependency in F has a single attribute for its RHS.
2. No dependency $X \to A$ in F can be replaced by a dependency $Y \to A$ where $Y \subseteq X$ and still has a set of dependencies equivalent to F.
3. We can't remove any dependency from F and still have a set of dependencies equivalent to F.

It is used to remove redundant FDs.

Q. $F = \{ A \to B,\ B \to C,\ AB \to C,\ A \to C \}$
Is it equivalent even if we remove $A \to C$?

Scanned by CamScanner

Date :
Page No :

Sol^n F: $A \to B$, $B \to C$, $AB \to C$, $A \to C$

$A^+ = ABC$ $AB^+ = ABC$

$B^+ = BC$

G: $A \to B$, $B \to C$, $AB \to C$

$A^+ = ABC$ $B^+ = BC$

$AB^+ = ABC$ Yes.

Extraneous attribute

It is the attribute which can be removed without changing the closure of F.D.

eg. $AB \to C$

$A \to C$ Here, B is extraneous.

Q. $\{AB \to C, A \to B\} = F$ and $G = \{A \to C, A \to B\}$ Find the extraneous attribute here.

Sol^n F: $AB \to C$, $A \to B$

$AB^+ = ABC$ ↖ same

$A^+ = ACB$ ↙

G: $A \to C$, $A \to B$

$A^+ = ABC$

∴ B is extraneous.

Q. Find minimal set. $F = \{A \to BC, B \to C\}$

Sol^n $A \to B$, $A \to C$] Decomposition Rule

$B \to C$

$\{A \to B, B \to C\}$] Transitivity

↓ minimal set

Q. $F = \{AB \to CD, BC \to D\}$ Find minimal set.

Sol^n $AB \to C$

$AB \to D$

$BC \to D$ $= \{AB \to C, BC \to D\}$

Scanned by CamScanner

11/3/19

Date :
Page No :

Q. Prove $AB \to CD, BC \to D = AB \to C, BC \to D$

Sol^n $AB \to C, AB \to D, BC \to D$

$AB \to C, AB \to D, BC \to D$ $\qquad$ $AB \to C, BC \to D$

$AB \to C$ ×

$\therefore AB \to CD, BC \to D = AB \to C, BC \to D.$

RHS is minimal set of LHS.

Q. Find the minimal cover of $A \to BC, B \to C, C \to B$ using Armstrong identities

Sol^n $A \to B$ (decomposition rule)

$A \to C$ "

$B \to C$

$C \to B = \{A \to B, B \to C, (C \to B), A \to C\}$ ×

$= \{A \to B, B \to C, C \to B, (A \to C)\}$ ×

Both are correct.

Q. $ABCD \to E, E \to D, A \to B, AC \to D$. Find minimal set.

Sol^n $ABCD \to E$ (Transitivity Rule)

$ACD \to E$ (as B is extraneous attribute as $A \to B$)

$AC \to E$ ($\because AC \to D$)

$\Rightarrow \{AC \to E, E \to D, A \to B\} \to$ minimal set

Q. $A \to BC, B \to C, A \to B, AB \to C$. Find minimal set.

Sol^n $\Rightarrow A \to B, A \to C, B \to C, A \to B, AB \to C$

$\Rightarrow A \to B, A \to C, B \to C, AB \to C$

$\Rightarrow A \to C, AB \to C, A \to B$

$\Rightarrow \{A \to C, A \to B\} \to$ minimal set

Q. ~~$AC \to E, E \to D, A \to B$~~ $A \to BC, B \to AC, C \to AB$. Find minimal set.

Sol^n ~~$AC \to D$~~ $A \to B, A \to C, B \to A, B \to C, C \to A, C \to B$

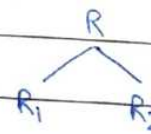
$A \to B, B \to C, B \to A, C \to AB$

$\{A \to B, B \to A, B \to C, C \to B\} \to$ minimal set

Scanned by CamScanner

Date :
Page No :

÷ Decomposition:



$R_1 \cup R_2 = R$

A relation R can be decomposed into a collection of relational schemas to eliminate some of the anomalies in original relation R.

(i) Lossless join decomposition: Let R be a relation having a set F of func. dependencies. The decomposition of R into R_1 and R_2 is lossless wrt F if $R_1 \bowtie R_2 = R$.

(ii) Attribute preservation: All the attributes of the original schema must appear in the decomposition, either in R1, R2 or both.

(iii) Dependency preservation:

eg. Enroll (sno, cno, date, roomno, instructor)
Enroll1 (sno, cno, date)
Enroll2 (date, roomno, instructor)

Q. Suppose we decompose a schema R (ABCDE) into R_1(ABC) and R_2(ADE). $A \to BC$, $CD \to E$, $B \to D$, $E \to A$.
Find if it is lossless decomposition or lossy.

Sol^n $R_1 \cap R_2 = A$
↳ A is candidate key so it is lossless decomposition.

Q. You are give a relation R (ABCD) and set of func. dependencies is F: $A \to B$, $C \to D$. Find if it lossless decomposition or lossy and if the dependencies are preserved or not. R_1(AB), R_2(CD).

Sol^n R_1 (AB) R_2(CD)
$R_1 \cap R_2 = \phi \to$ lossy decomposition
$A \to B$ belongs to R_1 and $C \to D$ to R_2.
So, dependencies are preserved.

Scanned by CamScanner

Date :
Page No :

Q. Find if the relations $R_1 = ABC$ and $R_2 = ADE$ are lossless / lossy
R(ABCDE)
$A \to CD,\ B \to D,\ CD \to E,\ E \to A$

Sol^n $R_1 \cap R_2 = A$

$A^* = ACDE \to$ super key
↓
all RHS of func. dependencies are being covered so it is super key.

∴ Decomposition is lossless.

lossless even if $R_1 \cap R_2 = R_1$ or R_2
Dependencies are not preserved. $(A \to CD)$

Q. Consider a relation schema $R = U, V, X, Y, Z$.
FD : $Z \to U,\ V \to Y,\ XY \to Z,\ U \to VX$
Determine if $R_1(UVX)$ and $R_2(UYZ)$ is lossy / lossless.

Sol^n $R_1 \cap R_2 = U$

$U^* = UVXYZ$
↳ candidate key

∴ Decomposition is lossless.
Dependencies are not preserved. $(V \to Y)$

Q. You are given a relation R (ABCD).
FD : $A \to B,\ A \to C,\ C \to D$; $R_1 = ABC$, $R_2 = CD$

Sol^n $R_1 \cap R_2 = C$

$C^* = CD = R2$

∴ Decomposition is lossless.
OR

	A	B	C	D	
R_1	∝	∝	∝	∝	// $C \to D$
R_2			∝	∝	

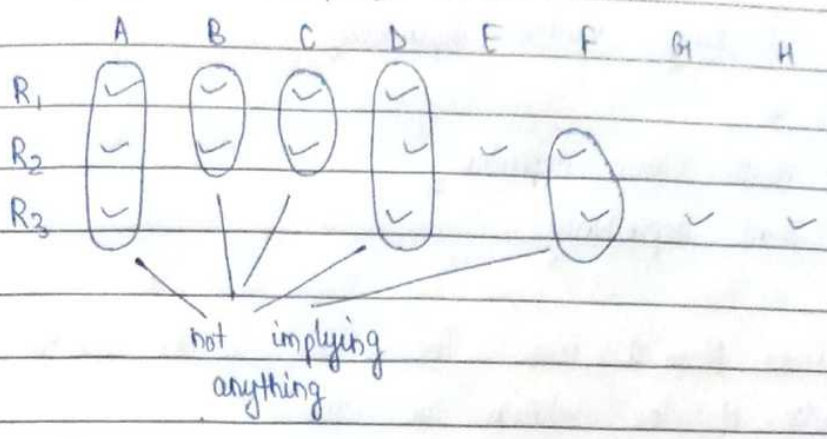
There's 1 row without any incomplete box so, it is lossless.

Scanned by CamScanner

Date :
Page No :

Q. You are given R (ABCDEFGH).
FD: AB → C, BC → D, E → F, GB → F, H → A, FG → H
R_1 (ABCD), R_2 (ABCDEF), R_3 (ABFGH)

Sol^n



This is lossy decomposition.

15/3/19

Normalization

It is a process of analysing given relational schema based on their functional dependencies and primary keys to achieve desired properties of minimising redundancy and hence minimising insertion, deletion and update anomalies.

Std (Name, Course, mobile, sub, Professor, Grade)
Key: Name, course
FD: Name → Mobile
Name → Sub
Name course → grade
Course → Prof

Update anomaly:
Insertion anomaly:
Deletion anomaly:

Normal form: Used to eliminate a reduced redundancy in relations;
A set of rules to perform decomposition

Scanned by CamScanner

Page No :

Types of normal forms:

1. 1NF: By default, our schema is in 1NF.
2. 2NF: ┐
3. 3NF: ├ - single valued dependency
4. BCNF: ┘
5. 4NF: Multi value dependency
6. 5NF: Joint dependency

1. 1NF (First Normal Form): The values in the domain of each attribute of the relation are atomic.
Only 1 value is associated to each attribute.

Eno	Ename	Contact
1	A	99, 98
2	B	199, 100

↓ 1NF

Eno	Ename	Contact
1	A	99
1	A	98
2	B	199
2	B	100

(atomic)

1NF has highest redundancy.

2. 2NF (Second Normal Form): It is based on the concept of fully functional dependency. A relation R is in 2NF if:
1. If is in 1NF.
2. Every non-prime attribute of R is fully functional dependent on the key R. / No non-prime attribute should be determined by the part of candidate key. / No partial dependency

eg. R (ABCD) C.K. = AB
Prime attributes AB
Non-Prime CD

Scanned by CamScanner

Date :
Page No. :

$AB \to C$ ✓ $\quad B \to C$ ×

$AB \to D$ ✓ $\quad A \to D$ ×

$AB \to CD$ ✓

Q. R(ABCD)

FD: $AB \to C$, $B \to D$ Find the normal form.

Sol^n $AB^+ = ABCD$ (CK)

$B^+ = BD$

$AB \to C$ ✓ (in 2NF)

$B \to D$ × (not in 2NF)

∴ Overall relation is not in 2NF. It is in 1NF.

Q. R(ABC)

FD: $A \to B$, $B \to C$ Find the CK and normal form.

Sol^n $A^+ = ABC$ (CK)

$B^+ = BC$

$A \to B$ ✓ (in 2NF)

$B \to C$ ✓ (in 2NF) (non prime → non prime)

- Attributes not present on RHS in any functional dependencies would always be a part of CK.

∴ R is in 2NF.

Q. R (ABCD)

FD: $AB \to C$, $A \to D$ Decompose it into 2NF.

Sol^n $AB^+ = ABCD$ (CK)

$AB \to C$ ✓ (in 2NF)

$A \to D$ × (not in 2NF)

$R_1(ABC)$ $AB \to C$ $\quad R_2(AD)$ $A \to D$ → in 2NF

Q. R(ABCDE)

FD: $AB \to C$, $A \to D$, $B \to E$ Decompose it into 2NF.

Scanned by CamScanner

Sol^n $AB^+ = ABCDE$ (CK)

$AB \to C$ ✓

$A \to D$ ✗

$B \to E$ ✗

$R_1(ABC)$ $R_2(AD)$ $R_3(BE)$

3. 3NF (Third Normal form): Dependent on transitive dependency. Most real time databases are normalised in 3NF.

A relation schema R is in 3NF if it satisfies:

1. It is in 2NF.
2. No non-prime attribute of R is transitively dependent on the key of R. / A relation is in 3NF if:

1. FD: $X \to Y$ satisfies any one of the following conditions:

(i) $X \to Y$ is a trivial FD i.e. Y is a subset of X.

(ii) $X \to Y$, then X is a superkey.

(iii) $X \to Y$, then $(Y - X)$ is a prime attribute.

NOTE: $X \to Y$ and $Y \to Z$ is allowed in 3NF.

X: SK, Y: SK (superkeys), Z: non-key

$X \to Y$ and $Y \to Z$ is not allowed in 3NF.

X: SK, Y and Z: non-keys

Q. R(ABC)

FD: $A \to B$ $B \to C$ R is in which normal form?

Sol^n $A^+ = ABC$ (CK)

$A \to B$ in 3NF (A is superkey)

$B \to C$ not in 3NF ($C - B = C$ not prime)

$A \to B$ 2NF

$B \to C$ 2NF

Overall relation is in 2NF.

Scanned by CamScanner

Date :
Page No :

Q. R (ABCDE)

FD: $AB \to C$, $B \to D$, $D \to E$ Decompose into 3NF.

Soln

$AB^+ = ABCDE$ (CK)

$AB \to C$ ✓ 3NF

$B \to D$ $D - B = D$ not prime × 3NF

↓ not a superkey

$D \to E$ $E - D = E$ not prime × 3NF

$R_1(ABC)$ $R_2(BD)$ $R_3(DE)$

18/3/19

Q. R(ABC)

FD: $AB \to C$ $C \to A$ Decompose into 3NF.

Soln

$B^+ = B$ ×

$AB^+ = ABC$ (CK)

$BC^+ = BCA$ (CK)

$AB \to C$ in 3NF

↳ CK

$C \to A$ not in 3NF as per II rule

↳ not a CK

$A - C = A$ ∴ $C \to A$ is in 3NF

↳ prime

Q. Create a relational schema R with primary key AB s.t. R is in 2NF but not in 3NF. R(ABCD)

Soln

$AB \to C$ ✓ in 2NF and 3NF

$C \to D$ ✓ in 2NF but not in 3NF

↓ not a Superkey

Scanned by CamScanner

18/3/19

UNIT-3

4) Boyce Codd Normal Form (BCNF):

It is a stricter version of 3NF. It is an extension of 3NF.

A relation R is in 3NF if:

1. $X \to Y$ is a trivial FD
2. $X \to Y$, X is a superkey

Q. R(ABC) F: $\{AB \to C, C \to A\}$

Find if R is in 3NF or BCNF.

Sol^n

$B^+ = B$

$AB^+ = ABC$ (CK)

$BC^+ = BCA$ (CK)

$AB \to C$ in BCNF

↳ superkey

$C \to A$ but $A - C = A$ is prime. ∴ This is in 3NF.

↳ not a superkey

So, R is in 3NF.

NOTE: Every BCNF is in 3NF but vice-versa is not true.

Q. R(ABCDEFGHIJ)

F: $\{AB \to C, A \to DE, B \to F, F \to GH, D \to IJ\}$

Decompose it into BCNF.

Sol^n

$AB^+ = ABCDEFIJGH$ (CK)

$AB \to C$ ✓

$R_1(ABC)$, $A \to DE$ $R_2(ADE)$, $B \to F$ $R_3(BF)$, $F \to GH$ $R_4(FGH)$

$D \to IJ$ $R_5(DIJ)$

- BCNF sometimes violates the dependency preservation.
- Because of BCNF decomposition, sometimes initial relation R CK is lost. That results in failure of dep. pres.
- If there is any dep. like Prop. subset of CK → Prop. subset of CK, dep. pres. is violated.

Scanned by CamScanner

- Let R consist of simple CK, then R is always in 2NF but may or may not be in 3NF or BCNF.
- If R consists of only prime attributes, then R is in 3NF but may or may not be in BCNF.
- If every attribute is a superkey, R is in BCNF.
- R with no trivial F.D., then R is in BCNF.
- If there is a binary relation R, it is always in BCNF.

Q. R(ABCDE) $F: \{AB \to C, B \to D, D \to E\}$
Decompose it into 3NF.

Sol^n $AB^+ = ABCDE$ (CK)

$AB \to C$ in 3NF

$B \to D$ not in 3NF

$D \to E$ not in 3NF

$R_1(ABC)$, $R_2(BD)$, $R_3(DE)$

Q. R(ABCDEFGHIJ) $F: \{AB \to C, AD \to GH, BD \to EF, A \to I, H \to J\}$ Decompose

Sol^n $AB^+ = ABI$ ×

$ABD^+ = ABDCGHEFIJ$ (CK)

$AB \to C$

↳ partial dep.

$R_1(ABCI) \to R_{11}(ABC), R_{12}(AI)$

↳ not in 3NF

$AD \to GH$ $R_2(ADGHEJ) \to R_{21}(ADGH), R_{22}(HJ)$

↳ Transitive dep.

$BD \to EF$

$R_3(BDEF)$

Since CK is not covered in any relation, so $R_4(ABD)$.

Scanned by CamScanner

Q. Decompose into BCNF. R(ABCDEFGHIJ)

F: { AB→C, B→F, A→DE, D→IJ, F→GH }

Sol^n AB^+ = ABCDEFGHIJ (CK)

AB→C Partial dep. ∴ not in 2NF

$R_1(ABC)$, $R_2(\underline{ADEIJ})_{A^+}$, $R_3(\underline{BFGH})_{B^+}$ → in 2NF

$R_{21}(ADE)$ $R_{31}(BF)$

$R_{22}(DIJ)$ $R_{32}(FGH)$

Q. R(ABCDEFGH)

F: { AB→C, AC→B, AD→E, B→D, BC→A, E→G }

Decompose into BCNF.

Sol^n $ABFH^+$ = ABFHCDEG ✓

$BCFH^+$ = BCFHADEG ✓

$ACFH^+$ = ACFHBDEG ✓

$R_1(ABC)$, $R_2(ACB)$, $R_3(ADE)$

$R_4(BD)$, $R_5(BCA)$

$R_5(EG)$

Q. R(ABCDEH)

F: {A→B, BC→D, E→C, D→A }

Identify the highest normal form.

Sol^n $BCEH^+$ = BCEHDA ✓

A→B ∴ not in BCNF

↳ not a superkey

D-BC = D → not prime ∴ not in 3NF

A→(B) ∴ not in 2NF

↓ ↳ prime

non-prime

This relation is in 1NF.

Scanned by CamScanner

25/3/19

Fourth Normal form (4NF): A relation R is in 4NF if:

(i) R is in 3NF and BCNF.

(ii) R contains no multi valued dependencies.

Multivalued dependency is a dependency where 1 attribute value is potentially dependent on more than 1 values.

eg.

Person (P)	Mobile (M)	food (F)
P_1	M_1	F_1
P_2	M_2	F_2
	M_3	F_3

$P \twoheadrightarrow M$

$P \twoheadrightarrow F$

Q. Consider the given table Students:

Name (N)	Computer (C)	Language (L)
Aman	Windows / Apple	Eng. / Hindi
Mohan	Linux	Eng. / Spanish

Sol.ⁿ $N \twoheadrightarrow C$

$N \twoheadrightarrow L$

2 tables (N, C) and (N, L)

Fifth Normal Form / Projected Join Normal Form) (PJNF): A relation R is in 5NF if:

(i) R is in 4NF.

(ii) It cannot be further non loss decomposed.

Join dependency: Let R be a relation schema and $R_1, R_2, \dots, R_n$ be the decomposition of R. R is said to satisfy join dependency iff $\pi_{R_1}(R) \bowtie \pi_{R_2}(R) \bowtie \pi_{R_3}(R) \bowtie \dots \bowtie \pi_{R_n}(R) = R$

↳ projection of R_1

OR

Every legal instance $\delta(R)$ = join of its projections on $R_1, R_2, \dots, R_n$

Scanned by CamScanner

eg.

Agent (A)	Company (C)	Product (P)
Aman	C_1	Pen drive
Aman	C_1	MIC
Aman	C_2	Speaker
Mohan	C_1	Speaker

$R_1(A,C)$ $R_2(A,P)$ $R_3(C,P)$

$(R_1 \bowtie R_2) \bowtie R_3$

Agent	Company	Product	
Aman	C_1	Pendrive	
Aman	C_1	MIC	
Aman	C_1	Speaker	X → extra tuples
Aman	C_2	Pen drive	X
Aman	C_2	MIC	Since there is no join dependency, we can't decompose it into 5NF.
Aman	C_2	Speaker	
Mohan	C_1	Speaker	

Q. R is in 5NF.

Pname (P)	Skill (S)	Job (J)
Aman	DBA	J_1
Mohan	Tester	J_2
Rohan	Programmer	J_3
Sohan	Analyst	J_1

Sol.n

R_1 $(P \bowtie S)$ ~~$\bowtie$~~ ~~J~~ R_2 $P \bowtie J$ R_3 $S \bowtie J$

Pname	Skill	~~Job~~
Aman	DBA	~~J_1~~
Mohan	Tester	
Rohan	Programmer	
Sohan	Analyst	

Pname	Job
Aman	J_1
Mohan	J_2
Rohan	J_3
Sohan	J_1

Skill	Job
DBA	J_1
Tester	J_2
Programmer	J_3
Analyst	J_1

Scanned by CamScanner

$(R_1 \bowtie R_2) \bowtie R_3$

Pname	Skill	Job
Aman	DBA	J_1
Mohan	Tester	J_2
Rohan	Programmer	J_3
Sohan	Analyst	J_1

$R_1 \bowtie R_2 \bowtie R_3 = R$

There is join dependency.

∴ Not in 5NF.

28/3/19

Q. (Book title, Author name, Book Type, Publisher, List Price, Author efficiency)

B. title → (Publisher)(Book Type)

Book Type → List Price

Author name → Author efficiency

Decompose it.

Sol.ⁿ

R_1 (Book title, Publisher, Book Type)

R_2 (Book Type, List Price)

R_3 (Author name, Author efficiency)

∴ 3 relations R_1 will be formed.

R_2

R_3

Q. 1) Evaluate the CK → M, MY, MC

2) Tell if it is 3NF or BCNF.

R(M, V, P, MP, C)

F = {M → MP, {M, Y} → P, MP → C}

3) Tell if the decomposition is lossy or lossless.

R_1(M, Y, P) R_2(M, MP, C)

Sol.ⁿ

$M^+ = MMPC$ ×

$MY^+ = MYPC$ ✓ (CK)

$MC^+ = MC$ ×

∴ MY is the candidate key.

Scanned by CamScanner

$R_1(M, Y, P)$ $R_2(M, MP, C)$

M is common attribute.

$M^+ = MMPC \longrightarrow$ satisfies R_2

∴ It is lossless decomposition.

$M \rightarrow MP$ ✓

$MY \rightarrow P$ prime → non-prime ×

∴ It is not in 2NF so it can't be in 3NF or BCNF.

Scanned by CamScanner

29/3/19

UNIT-4

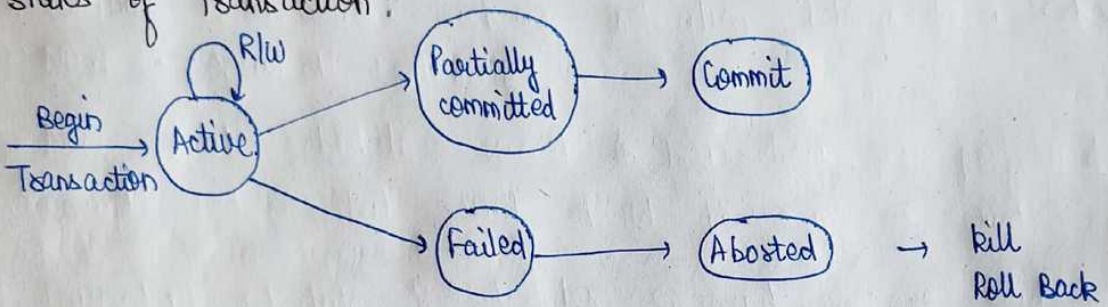
Transaction management

Transaction is a logical unit of DB processing that must be completed in its to ensure its correctness.

ACID:

1. Atomicity - not partially completed i.e. all or none
2. Consistent - correctness
3. Isolation - executing independently
4. Durability - updation ; permanent

States of Transaction:



Concurrent execution:

Executing more than one transaction at the same time using the interleaving technique.

Advantage:

1. ↑sed CPU utilisation: If CPU executed more than one transaction at same time, utilisation ↑ses. Also, parallel take both I/O and CPU activity this ↑ses CPU throughput also.

   Throughput - average no. of completions in given time
2. Better average response time
3. Interleaved execution of short transaction among long a short transaction executed first.
4. Better turnaround time (time blw submission and completion)
5. Better average waiting time
6. Better disk utilisation / resource utilisation

Scanned by CamScanner

Problems: W: write

1. Lost update problem (W-W conflict):

Eg.

T_1	T_2
R(A)	R(A)
A = A - 50	A = A + 100
950	1100
Update is lost → W(A)	W(A)

A lost update problem occurs when 2 transaction access the same data item and have their operations in such a way that makes the value of some data item incorrect.

2. Dirty Read Problem (W-R conflict / Write Read (Temporary update):

T_1	T_2	
R(A)		
A = A + 20		
120 W(A)	R(A)	120
	A = A + 10	130
R(B)	W(A)	130
failed	commit	

3. Unrepeatable Read:

T_1	T_2
R(A)	
	R(A)
	A = A + 200
R(A)	W(A)

4. Incorrect Summary Problem:

T_1	T_2
	sum = 0
	R(A), sum += A
	R(Y)
	sum += Y
R(Y)	
Y = Y + 100	
W(Y)	

Scanned by CamScanner

÷ Schedule: Ordering of operations on a transaction in a chronological order ($T_i \rightarrow T_j$)

Total no. of schedules = n!

Types:

1. Complete schedule: If transaction committed or failed.
2. Serial schedule: Transactions executed serially.
   A schedule that contains either an abort or commit for each transaction whose actions are listed in it.
   A complete schedule must contain all the actions of every transaction that occur in it.
   (It consists of serial of schedule sequence of instructions from various transactions where instruction belongs to a schedule.)
   NOTE → consistent schedule
3. Non-serial schedule: Transactions are performed in non-serial order but result should be same as serial.
4. Equivalent schedule: 2 schedules S_1 and S_2 are said to be equivalent if they produce the same final database state.
   1. Result equivalent schedule 2. Conflict equivalent schedule
   1. If it produces the same final database state for same initial value of data.

S_1	S_2
R(A)	R(A)
A → A+10	A = A × 1.1
W(A)	W(A)
A = 110	A = 110

Conflicts are said to be there iff all the conflicting operations in both the schedules occur in same order.

5. Serializable schedule: A non-serial schedule that is equivalent to some serial schedule.
6. Recoverable schedule: Each pair of transactions T_i, T_j s.t. T_j reads a data item written by T_i. Then the commit operation of T_i should appear before that of T_j.

Scanned by CamScanner

T_1	T_2
R(A)	
A=A+50	
W(A)	
	R(A)
	A=A+100
	W(A)
R(B)	
W(B)	
commit	commit

7. Cascadeless schedule: For each pair of transactions T_i, T_j, T_j reads a data item written by T_i, the commit operation of T_i should appear after the write operation of T_j.

T_1	T_2	T_3
R(A)		
W(A)		
	R(A)	
commit	W(A)	
		R(A)
	commit	W(A)
		commit

8. Strict schedule: If a value written by transaction can't be read or over-written by another transaction until the transaction is either aborted or committed.

Every strict schedule is recoverable and cascadeless.

9. View serializability: 2 schedules S and S' are view

1. For each data item Q if T_i reads an initial value Q in S then T_i in S' also reads an initial value Q.
2. If T_i executes read Q in S s.t. value was produced for T_i and T_i in schedule S' also reads value of Q produced by T_j.
3. For each data item Q, the transaction that performs final write in schedule S must also perform final write in S'.

A schedule is view serializable if it is view equivalent to a serial schedule.

Every conflict serializable schedule is view equivalent and thus view serializable but vice versa is not true.

Scanned by CamScanner

1/4/19

Concurrency control

It is the process of managing simultaneous execution of transactions to ensure the serializability of transaction.

Purpose: 1. To enforce isolation
2. To preserve database
3. To resolve read-write, write-write conflicts.

Lock Based Protocols:

A lock guarantees exclusive use of data item to a current transaction.

→ To access data item (lock is required)

→ Lock is released

Types of locks:

1. Shared lock: Lock S (Read)
2. Exclusive lock: Lock X (Read-write)

Lock compatibility matrix:

	S	X
S	T	F
X	F	F

Request → concurrency control manager
↳ (either lock is granted or denied)

Conversion of locks:

1. Upgrading: S → X
2. Downgrading: X → S

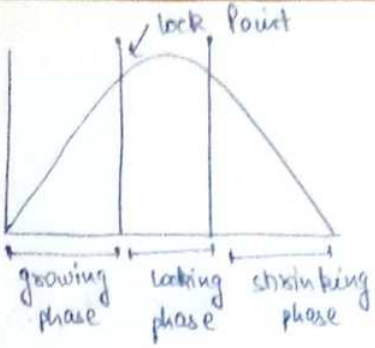
Lock table → manages the transaction locks

Two Phase locking Protocol:

1. Growing Phase - locks are acquired not released; also called expanding
2. Shrinking Phase - locks are released not acquired

The protocol assumes that a transaction can only be in 2 phases.

Scanned by CamScanner



Lock point : A point where all the locks it may need has been acquired

3/4/19

Disadvantages:

1. Holding the lock unnecessarily.
2. Locking before required
3. Penalty to other transactions

Variations:

1. Strict 2 Phase lock Protocol - for exclusive locks ; cascading rollback
2. Rigorous 2 Phase lock Protocol - for shared and exclusive locks ; commit
3. Static 2 Phase lock Protocol - cascading rollback ; deadlock free

→ Binary locks - It has 2 states - lock and unlock. If lock (A)=1 then item A can't be accessed and the transaction is forced to wait. Else if lock (A)=0 , item A can be accessed.

Disadvantages:

→ easy to implement but restricted to yield optimal concurrency conditions

→ DB must not be allowed to read same DB objects

Q. Given 3 schedules S_3 is $R_1(X), R_2(Z), R_1(Z), R_3(X), R_3(Y), W_1(X), C_1(X), W_3(Y), C_3, R_2(Y), W_2(Z), W_2(Y), C_2$

S_4 is $R_1(X), R_2(Z), R_1(Z), R_3(X), R_3(Y), W_1(X), W_3(Y), R_2(Y), W_2(Z), W_2(Y), C_1, C_2, C_3$,

S_5 is $R_1(X), R_2(Z), R_3(X), R_1(Z), R_2(Y), R_3(Y), W_1(X), W_1(X), W_2(Y), W_2(Y), C_3, C_2$

Tell if these schedules are strict, cascadeless or recoverable.

Scanned by CamScanner

Q. $T_1 (R_1(X), R_1(Z), W_1(X))$

$T_2 (R_2(Z), R_2(Y), W_2(Z), W_2(Y))$

$T_3 (R_3(X), R_3(Y), W_3(Y))$

S_1: $R_1(X), R_2(Z), R_1(X), R_3(X), R_3(Y), W_1(X), W_3(Y), R_2(Y), W_2(Z), W_2(Y)$

S_2: $R_1(X), R_2(Z), R_3(X), R_1(Z), R_2(Y), R_3(Y), W_1(X), W_2(Z), W_3(Y), W_2(Y)$

Draw the serializability graph for S_1 and S_2.

Tell if S_1 and S_2 are serializable.

Sol.

S_1

T_1	T_2	T_3
R(X)		
	R(Z)	
R(X)		
		R(X)
		R(Y)
W(X)		
		W(Y)
	R(Y)	
	W(Z)	
	W(Y)	

S_2

T_1	T_2	T_3
R(X)		
	R(Z)	
		R(X) → conflicting
R(Z)		
	R(Y)	
		R(Y)
W(X)		
	W(Z)	
		W(Y)
	W(Y)	

S_1: $R_1(X)$ $R_2(Z)$ $R_1(X)$ $R_3(X)$ $R_3(Y)$ $W_1(X)$ $W_3(Y)$ $R_2(Y)$ $W_2(Z)$ $W_2(Y)$

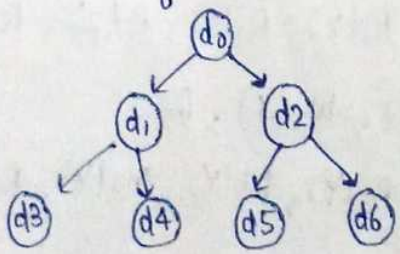
S_1 is serializable, S_2 is not.

5/4/19

Graph Based Protocol

This protocol ensures conflict serializability.

Tree Protocol:



Before locking, ensure that its parent is locked.

Scanned by CamScanner

Advantages of Tree Protocol:

1. Unlocking earlier so there is shorter waiting time for other transactions and hence ↑ed concurrency.
2. No rollback required

Disadvantages:

1. Parent data item has to be locked even if not needed.

Time Stamp Protocol:

1. Counter
2. System time

Time stamp values:

1. W- time stamp (latest time when a transaction wrote something)
2. R- time stamp (latest time when a transaction read something)

Time stamp ordering protocol:

1. T_i issues Read(Q) $T_s(t_i) < W(Q)$
2. T_i issues Write(Q) $T_s(t_i) \geq W(Q)$ and $T_s(t_i) \geq R(Q)$

Thomas Write Rule:

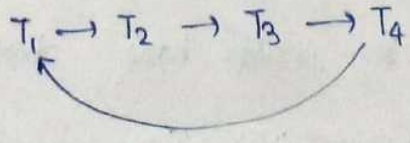
It is a modified version of Time stamp ordering protocol.

$T_s(t_i) < R(Q)$ → write is rejected and transaction is rolled back.

$T_s(t_i) < W(Q)$

Deadlock

A deadlock occurs when all transactions are waiting for each other to get completed.



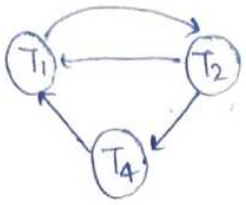
Scanned by CamScanner

1. Deadlock Prevention
2. Deadlock Detection and recovery

$T_i \rightarrow T_j$

T_i is waiting for resource data item held by T_j.

Transactions	Data items	Lock mode	
T_1	Q	S	incompatible mode
		X	
T_2	P	X	
T_3	Q	S	
		X	
T_4	P		



wait-for graph

If a cycle exists, there is a deadlock.

Deadlock Prevention: ensures that system never goes into a deadlock

1. Mutual exclusion - 2 transactions should not access same data item simultaneously.
2. Hold and wait
3. No preemption
4. Circular wait

Deadlock prevention methods:

1. Tree Protocol
2. Pre-emption and transaction rollback

Use of time stamp in deadlock prevention:

1. Wait-Die scheme: $T_s(t_i) < T_s(t_j)$

   T_i is older than T_j so T_i is made to wait.

   Had it been younger, T_j would have aborted T_i.

2. Wound-wait scheme: $T_s(T_i) < T_s(T_j)$ [T_i is older than T_j]

   T_j is aborted.

Scanned by CamScanner

Recovery from deadlock:

One or more transactions are rolled back to break the deadlock.

1. Selection of victim - The transaction involving min. cost
   (i) How long the transaction has been executing
   (ii) How many data items it can hold.
   (iii) How many data items are needed to complete the transaction
   (iv) How many transactions are involved in rollback

Scanned by CamScanner